FC7P01NI MSc Project

Level 7 – Project

Deep Learning Approaches to Sentiment Analysis in Romanised Nepali English Code-Switched Text

Assessment Type

Semester

2025 Autumn

Student Name: Kshitiz Bhujel

London Met ID: 19033549

College ID: np01ms7a230040

Assignment Due Date: Friday, January 23, 2026

Assignment Submission Date: Friday, January 23, 2026

Supervisor Name: Dr Bal Krishna Bal

Word Count (15000): 13500

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submissions and a mark of zero will be awarded.

Table of Contents

List of Figures 6
List of Tables 7
Abstract 8
1. Introduction 1
1.1 Background 1
1.2 Problem Statement 3
1.3 Rationale 5
1.4 Research Question 6
1.5 Research Aims and Objectives 6
1.6 Scope and Limitation 7
1.7 Dissertation Structure 8
2. Literature Review 9
2.1 Sentiment Analysis in Low-Resource and South Asian Languages 9
2.2 Multilingual Pre-Trained Transformers for Low-Resource Language 11
2.3 Hyperparameter Tuning in Deep Learning 12
2.4 Identification Research Gaps 13
2.5 Synthesis Matrix 14
3. Theoretical Foundation 15
3.1 Conceptual Foundation: 15
3.1.1 Sentiment Analysis 15
3.1.2 Code-Switching and Romanization 16
3.1.3 Challenges in Code-Switched Romanized Nepali-English Sentiment Analysis 17
3.2 Technical Foundation 18
3.2.1 Generic Sentiment Analysis Pipeline 18
3.2.2 Bidirectional Long Short-Term Memory (BiLSTM) 19

3.2.3 Transformer-Based Models: mBERT and XLM-RoBERTa 21
3.2.4 Hyperparameters and Model Training..... 23
4. Research Methodology 25
4.1 Research Design 25
4.2 Dataset 25
4.2.1 Data Source and Collection 25
4.2.2 Data Statistics..... 26
4.2.3 Train/Validation/Test..... 26
4.3 Preprocessing..... 26
4.3.1 Text cleaning..... 26
4.3.2 Tokenization..... 27
4.4 Model Development..... 28
4.4.1 BiLSTM Architecture 28
4.4.2 mBERT Architecture 29
4.4.3 XLM-R Architecture..... 29
4.5 Hyperparameter Tuning Strategy..... 30
4.6 Evaluation Metrics and Strategy 31
4.6.1 Standard Metrics 31
4.6.2 RQ1 Evaluation 31
4.6.3 RQ2 Evaluation: Cross-Lingual Consistency..... 32
1. Implementation, Results, and Discussion 33
5.1 Experimental Setup..... 33
5.2 Implementation Details 33
5.2.1 Training Procedure 36
5.3 Model Performance on the Main Dataset (RQ1) 37
5.3.1 Hyperparameter Tuning Results 37

5.3.2 Test Set Performance 38
5.3.3 Training Curves and Loss Analysis 39
5.3.4 Discussion of RQ1 Results 41
5.4 Cross-Lingual Consistency Analysis (RQ2) 42
5.4.1 Per-variant Accuracy and Confidence 43
5.4.2 Consistency Rate 44
5.4.3 Discussion of RQ2 Findings 45
5.5 Error Analysis 46
5.5.1 Overview of Failure Cases 46
5.5.3 Error Type 2: Neutral Class Confusion 48
5.5.4 Error Type 3: Code-mixed Specific Failures 50
5.5.5 Error Type 4: Overconfident Misclassifications 51
5.5.6 Summary of Error Patterns 52
2. Conclusion and Recommendations for Future Work 53
6.1 Summary of Findings 53
6.2 Research Contribution 55
6.3 Limitations: 56
6.4 Recommendation and Future Work 58
6.4 Final Remark 60
References 61
Appendices 1
Appendix A: Dataset Annotation Guidelines 1
Appendix B: Hyperparameter Tuning Full Results 2
B.1 BiLSTM with Attention – Complete Grid Search Results 2
B.2 Mulitlingual BERT (mBERT) – Completed Tuning Results 2
B.3 XLM-RoBERTa (XLM-R) – Complete Tuning Results 3

Appendix C: Loss Curves of All Models 4
C.1 BiLSTM with Attention – Training Dynamics 4
C.2 mBERT Training and Validation Curves 5
C.3 XLM-R Training and Validation Curves 6
Appendix D: Confusion Matrices 7
D.1 BiLSTM with Attention Confusion Matrix 7
D.2 mBERT Confusion Matrix 8
D.3 XLM-R Confusion Matrix 9
D.4 Cross-Lingual Consistency – XLM-R Per-Variant Confusion Matrices 10
Appendix E: Logbook 13

List of Figures

Figure 1 Sentiment Analysis Pipelines 18
Figure 2 BiLSTM Architecture 28
Figure 3 mBERT Architecture 29
Figure 4 XLM-R Architectures 29
Figure 5 Training vs Validation Curve (BiLSTM + Attention) 39
Figure 6 Trainig vs Validation Curve (mBERT) 40
Figure 7 Trainig vs Validation Curve (XLM-R) 40

List of Tables

Table 1 Synthesis Matrix 14
Table 2 Example Romanised Nepali 16
Table 3 Dataset Statistics 26
Table 4 Hyperparameter Tuning Strategy 30
Table 5 Hyperparameter Search Space 1 34
Table 6 Hyperparameter Search Space 2 35
Table 7 Hyperparameter Search Space 3 36
Table 8 Hyperparameter Tuning (BiLSTM 37
Table 9 Hyperparameter Tuning (mBERT) 37
Table 10 Hyperparameter Tuning (XLM-R) 38
Table 11 Optimal Model Test Performance 38
Table 12 Per-Variant Accuracy and Confidence Score 43
Table 13 Complete Negation Misinterpretation 46
Table 14 Partial Negation Failure 47
Table 15 Neutral Class Confusion 48
Table 16 Misclassification 48
Table 17 Positive Drift for Neutral Sentiment 49
Table 18 Code-mixed Underperformance 50
Table 19 Code-mixed Misinterpretation 50
Table 20 Overconfident Misclassification 51

Abstract

To understand social media sentiment, sentiment analysis is essential. Most study focusses on well-resourced English or other languages in their native scripts. South Asian online chatters mix languages. Nepalis write their language in Roman letters instead of scripts. YouTubers who mix Romanised Nepali and English love this. This is frequent, but few studies have examined how well sentiment analysis models operate with this language combination and writing style.

This study analyses three deep learning methods for sentiment classification on Romanised Nepali-English code-mixed YouTube comments: BiLSTM with Attention (a sequence model built from scratch), multilingual BERT (mBERT), and XLM-RoBERTa (XLM-R), both pre-trained transformer models. We manually sorted YouTube comments into favourable, negative, and neutral. We then carefully modified hyperparameters for each architecture and monitored how the models trained using limited code-mixed data.

All three models validated best in the second training epoch. This explains why restricted datasets require early stopping. Best overall on the main test set was XLM-R with a weighted F1-score of 82.3%. BiLSTM placed last with 67% and mBERT second with 81.5%. The training curves demonstrated varied model learning. XLM-R showed stable, controlled learning with limited overfitting (a train-validation gap of 0.19), BiLSTM had moderate overfitting (0.38), and mBERT had severe late-stage degradation, with validation loss growing sharply after the ideal stopping point.

When tested on sentences that signify the same thing in three languages, the best model (XLM-R) got 86.67% of English text right but only 60% of Romanised Nepali text and 53.33% of code-mixed text. More than 26 percentage points difference. Only 46.67% of sentence triplets predicted correctly in all three languages. A thorough error investigation revealed four systematic issues: The model missed Nepali negation markers like "bhayena" and "naramro," predicting the opposite sentiment; neutral Nepali expressions like "thikai" and "milcha" were misclassified as negative or positive; code-mixed sentences performed worse than single-language versions; and the model was dangerously overconfident in its wrong Romanised Nepali predictions, sometimes reaching 85 - 97% confidence while being completely wrong.

These data suggest that pre-training improves current multilingual transformers, especially XLM-R, which has a 100-language training corpus. Even when representing the same language, they still favour high-resource languages and struggle with non-standard scripts. The research includes a novel annotated dataset for this understudied language pair, a meticulous three-way architectural comparison with hyperparameter tuning results, an innovative triplet-based evaluation method for cross-lingual consistency, and extensive documentation of Romanised Nepali-English sentiment analysis failure patterns.

As code-switching grows in South Asian digital spaces, establishing NLP systems that operate as effectively for multilingual users as they do for English-only users is not just a technical issue; it is about making AI development truly inclusive. Future research should focus on negation and neutral expression data collection, Romanised Nepali spelling normalisation, larger language models, few-shot learning, and applying these methods to other South Asian language pairs facing similar challenges.

Sentiment analysis, code-mixing, Romanised Nepali, multilingual transformers, XLM-RoBERTa, BiLSTM with attention, low-resource NLP, YouTube comments, cross-lingual consistency

1. Introduction

1.1 Background

In Nepal, users commonly mix Nepali, Romanised Nepali, and English on social media to freely express their opinions, emotions, and everyday experiences. While this code-switch communication reflects real-world language practices, its automatic analysis remains challenging and underexplored, especially for sentiment analysis in low-resource settings like Nepali (Sameer Sitoula, 2025) (Niraj Pahari, 2023). Sentiment mining/opinion mining, also known as sentiment analysis, is a core area of natural language processing that focuses on identifying the emotional tone and polarity expressed in textual content. With the rapid growth of social media, online reviews, and user-generated content, sentiment analysis has gained significant importance for organisations, governments, and researchers in Nepal, as it has a deeper understanding of public opinion related to products, policy, and broader social issues expressed across multilingual and code-mixed digital space (Sanjeepan Sivapiran, 2023).

South Asian nations, particularly Nepal, have experienced remarkable expansion in online interaction through platforms such as Facebook, YouTube, Twitter (X), and TikTok recently. A substantial share of this digital discourse occurs through code-switched communication, where individuals blend two or more linguistic systems within a single conversation or even an individual's sentence. Within Nepal, Nepali is the official language of Nepal and is spoken by 13.71 million speakers in Nepal (Rauniyar, 2025). While standard Nepali is written in Devanagari, many users now write Nepali with the Roman alphabet on social media and e-commerce platforms, which creates Romanised Nepali texts that mix Nepali words with English spellings and vocabulary.

Examples include the phrase:

> "Ma Khana khanxu, it tastes amazing!" (I will eat; it tastes amazing) OR
>
> "Timi kati dherai ramri, you deserve the best" (You are so beautiful, you deserve the best), demonstrating intra-sentential language alternation.

Romanised Nepali denotes Nepali language content transcribed using Latin alphabetic characters instead of the conventional Devanagari writing system. This transcription

method is characteristically casual, lacks uniform standards, and demonstrates considerable orthographic variation. A single Nepali term meaning “good” might appear as “ramro”, “ramroo”, or “ramroo”, “sahi xa” based on individual writer preferences, illustrating the inconsistency challenge (Chahil Choudhary, 2024) (Manish Pandey, 2025). The differences in spelling make it challenging for regular natural language processing systems to work well, as they usually rely on consistent language data from well-supported languages.

These Romanised and mixed texts show rich morphology, variable spelling, and informal style, making automatic processing and sentiment analysis more challenging than for well-resourced languages (Sameer Sitoula, 2025) (Abhash Pradhananga, 2023). Code-mixed or code-switched communication has complicated grammar and structure, along with a wide range of words from two or more different languages. The proliferation of social media and digital communication channels has encouraged multilingual speakers to routinely alternate or merge languages within identical sentences or conversational exchanges, a linguistic behaviour widely recognised as code-mixing or code-switching (Supriya Chanda, 2024).

Code-mixed or code-switched has complex grammar and parsing structure and a large vocabulary across two or more different languages (Shuyue Stella Li, 2022). With the rise of social media and online communication, multilingual speakers often switch or mix languages within the same sentence or conversation, a behaviour commonly referred to as 'code-mixed' or 'code-switching' (Shuyue Stella Li, 2022) (Niraj Pahari, 2023). Code-mixed texts usually combine complex sentence structures and large vocabulary from multiple languages and are mostly found in informal spaces such as chats, comments, ratings, and posts, where they are difficult to collect and annotate, so they are often treated as low-resource data (Shuyue Stella Li, 2022) (Niraj Pahari, 2023).

For Nepali-English users in particular, social media has created a growing stream of code-switched comments and reviews, but there are still limited works and few datasets targeting sentiment analysis for this specific language pair (Sameer Sitoula, 2025) (Shuyue Stella Li, 2022).

1.2 Problem Statement

As social media networks, digital review systems, and user-generated content continue to grow around the world, automated sentiment detection has become an important analytical tool. The ability to detect emotional orientation and textual polarity in online discourse is of enormous use to businesses, governments, and researchers who want to record public opinions as they are expressed in comments and evaluations (Chahil Choudhary, 2024) (Sanjeepan Sivapiran, 2023).

YouTube has evolved into a dominant communication channel where Nepali-speaking audiences express viewpoints, immediate responses, and emotional states through comment sections that routinely blend Romanized Nepali with English elements in the same written utterance, producing an unplanned and colloquial code-mixed linguistic context. (Supriya Chanda, 2024) However, notwithstanding the accelerating volume of such user-generated material, current sentiment classification systems engineered for single-language or conventional-script input fail to accommodate the distinctive attributes of Romanized Nepali-English code-mixed commentary, which manifests irregular orthography, unpredictable language alternation behaviours, and minimal contextual signals (Chahil Choudhary, 2024) (Niraj Pahari, 2023).

As mentioned, Romanized Nepali doesn't follow standard spelling rules, leading to uneven word usage and grammar changes that interfere with regular methods for breaking down text and creating word representations. (Niraj Pahari, 2023)Second, publicly accessible annotated corpora for sentiment analysis predominantly comprise monolingual English collections, monolingual Nepali materials in Devanagari format, or datasets addressing alternative language combinations such as Hindi-English, while labelled resources specifically targeting code-switched Romanized Nepali-English remain exceptionally scarce (Niraj Pahari, 2023).

Third, most pretrained lexical embeddings and even modern multilingual transformer models were mainly built using standard language inputs and well-known writing systems, making it unclear how well they can understand the meaning in Romanized, informal, code-switching text. Fourth, there has not been a thorough comparison of different types of models, from traditional sequential neural networks to modern cross-lingual

transformer frameworks—for classifying sentiment in Romanized Nepali-English code-switched text (Supriya Chanda, 2024).

Additionally, prior research efforts addressing Nepali sentiment analysis have concentrated primarily on Devanagari-script materials or different approach on romanized-nepali and English text, thereby creating a fundamental knowledge gap regarding optimal configuration and tuning strategies for deep learning architectures applied to the Romanized code-mixed scenarios.

The report addresses these multifaceted limitations through systematic empirical evaluation of BiLSTM with Attention mechanisms, multilingual BERT (mBERT), and XLM-RoBERTa (XLM-R) architectures applied to Romanized Nepali-English YouTube comment sentiment data, implementing targeted hyperparameter optimization procedures to determine which architectural configuration and parameter settings yield optimal classification accuracy for this authentic and demanding analytical scenario.

1.3 Rationale

YouTube comments represent a rich but under-explored source of public opinion in Nepal, where user's express sentiments about diverse topics ranging form entertainment and politics to social issues, predominantly using informal Romanized Nepali mixed with English vocabulary and structures (Sanjeepan Sivapiran, 2023).

Understanding sentiment in these comments is valuable for content creators, businesses, policymakers and researchers who seek to monitor audience reactions, track public moods during important events, and gather feedback on products or services within the Nepali-speaking community (Sameer Sitoula, 2025) (Niraj Pahari, 2023) (Niraj Pahari, 2023).

However, the spontaneous code-mixing behaviour that makes these comments natural for multilingual speakers also makes them extremely difficult for traditional natural language processing systems that rely on clean, monolingual input or standard orthography (Niraj Pahari, 2023).

Earlier methods for analyzing sentiment in Nepali mostly used traditional machine learning techniques with bag-of-words or TF-IDF features, which can't understand the context needed to interpret code-mixed comments where feelings might depend on subtle language switches or romanization patterns (Shuyue Stella Li, 2022).

Recent studies have initiated the examination of deep learning models for code-mixed sentiment analysis in various language pairs, including Hindi-English and Tamil-English. However, these results are not directly applicable to Romanized Nepali-English YouTube comments due to linguistic disparities, script variations, and the distinctive patterns of code-mixing within the Nepali context (Abhash Pradhananga, 2023).

Moreover, many existing works apply pre-trained transformers or sequence models with default or minimally adjusted hyperparameters, without systematically exploring how tuning learning rates, dropout rates, batch sizes, warmup steps, and other configuration choices affects model performance on specific low-resource data Moreover, many existing works apply pre-trained transformers or sequence models with default or minimally adjusted hyperparameters, without systematically exploring how tuning

learning rates, dropout rates, batch sizes, warmup steps, and other configuration choices affects model performance on specific low-resource data (Gibson Nkhata, 2025)

1.4 Research Question

This dissertation is guided by two primary research questions:

1. How do different deep learning architectures perform on sentiment analysis of code-switched Romanized Nepali–English text?
2. How does the use of code-mixed data vs. Romanized Nepali data impact the accuracy of the sentiment analysis models?

1.5 Research Aims and Objectives

This research tackles the identified challenges through the following goals:

- To build and assess how effective a deep learning architecture is for determining sentiment in Romanized/transliterated Nepali and English code-mixed text, while examining how consistently these models perform across different language combinations.
- To create and train three distinct neural network approaches: a bidirectional Long Short-Term Memory (BiLSTM) network built from scratch, mBERT from Transformer (mBERT), and Cross-lingual Language Model RoBERTa (XLM-R).
- To investigate how the top-performing architecture processes sentences with identical meaning expressed in Romanized Nepali or mixed formats by examining prediction confidence levels and output stability.

1.6 Scope and Limitation

Scope: This research mainly focuses on sentence-level sentiment classification using three emotional categories: positive, negative, and neutral. The primary textual material consists of casual user-contributed content collected from social media environments, especially YouTube comment sections. The linguistic focus encompasses Romanized Nepali and English, with particular attention to language-mixing patterns between the two languages. Three distinct neural network architectures are used for comparative assessment: BiLSTM, mBERT and XLM-R.

Limitation: Several limitations is defined of this study. The dataset size is limited to 1000 manually labeled sentences. Annotation procedures involve a limited number of human annotators, potentially introducing consistency variations in label assignment (Pahari, 2023).

1.7 Dissertation Structure

This report is organized as follows:

- Chapter 1 provides the introduction to my dissertation, the problem statement, and a brief overview of my report.
- Chapter 2 provides the theoretical foundation for sentiment analysis code-switching and the deep learning models.
- Chapter 3 reviews related literature on sentiment analysis, code-switched NLP, and multilingual models
- Section 4 describes the research methodology, including dataset preparation, preprocessing, model architecture, hyperparameter tuning, and evaluation strategy.
- Section 5 presents the implementation details, experimental results, performance comparisons, and detailed analysis of model behavior.
- Section 6 concludes a detailed analysis of the model's with a summary of findings, contributions, limitations, and recommendations for future work.

2. Literature Review

2.1 Sentiment Analysis in Low-Resource and South Asian Languages

South Asian language sentiment analysis has witnessed notable expansion over the past decade, though these languages continue to face significant resource constraints when compared to dominant languages such as English. Initial investigations into Hindi sentiment classification relied heavily on manually engineered linguistic features paired with conventional machine learning algorithms, including Support Vector Machines and Naive Bayes classifiers (Rajesh Piryani, 2020).

Contemporary research has increasingly adopted deep neural architectures, yielding promising results for code-mixed sentiment detection. Singh and Lefever's 2020 study examined Hinglish (a mix of Hindi and English) Twitter posts using cross-lingual word embedding techniques, demonstrating that multilingual vector spaces can effectively capture meaning across different languages. Their experiments confirmed that contextually aware embeddings surpass traditional frequency-based feature representations when processing casual social media language. (Pranaydeep Singh, 2020) Around the same period, Saroj and Pal contributed to the Dravidian Codemix shared task during FIRE2020, applying BERT-base models for analysing sentiment in multilingual code-mixed content (Anita Saroj, 2020). Their approach achieved competitive results by adapting pretrained multilingual language models to Dravidian language mixtures, demonstrating these transformer architectures' capability to accommodate the morphological richness and orthographic inconsistency characteristic of social media discourse (Anita Saroj, 2020).

Das's 2021 study investigated Hinglish sentiment recognition by contrasting various recurrent neural network designs incorporating attention mechanisms. Key findings indicated that attention-enhanced architectures more effectively pinpoint emotionally charged tokens within code-switched utterances where linguistic alternation influences affective expression (Das, 2021). Building on this foundation, Kumar and Pandey explored FNet and FastFormer models in 2022 for code-mixed sentiment and emotion categorization(Anuj Kumar, 2022). Their research established that computationally efficient transformer variants can match conventional BERT performance while

demanding substantially fewer processing resources, a critical consideration for low-resource language contexts (Anuj Kumar, 2022).

For Nepali specifically, research remains exceptionally sparse. Most existing Nepali sentiment analysis studies focus exclusively on Devanagari-script text using traditional machine learning approaches with TF-IDF features, which fail to capture contextual nuances. Critically, Pahari and Kazutaka (2023) conducted pioneering research examining language preferences for sentiment expression among Nepali-English bilingual speakers on social media. (Niraj Pahari, 2023) Their findings revealed that Nepali speakers tend to express negative emotions preferentially in their first language (Nepali) while adopting English for neutral or positive sentiments, suggesting that code-mixing patterns carry meaningful sentiment signals. (Niraj Pahari, 2023) This linguistic insight has profound implications for sentiment classification models, as language choice itself becomes a feature for prediction (Niraj Pahari, 2023).

What is missing: While Hinglish and Dravidian code-mixed sentiment analysis has progressed, systematic investigation of Romanized Nepali-English mixed text remains essentially absent. There is a significant knowledge gap about how computational models handle informal, spelling-inconsistent Romanised Nepali blended with English because current Nepali sentiment research focuses on standardised Devanagari orthography rather than Romanised alternatives that predominate on YouTube and similar social platforms (Niraj Pahari, 2023).

2.2 Multilingual Pre-Trained Transformers for Low-Resource Language

Pre-trained multilingual language models have changed the game for low -resource languages by letting researchers transfer knowledge from languages with lots of training data. Multilingual BERT (mBERT), trained on 104 languages, including Hindi and Nepali, works surprisingly well on cross-language tasks, even for languages that didn't get much representation in their training. Studies show that mBERT somehow learns to align different languages representations, enabling it to apply sentiment knowledge learned from one language to another (Hu, 2023).

XLM-RoBERTa (XLM-R) came along as a more robust version, trained on 100 languages using massive 2.5TB dataset form CommonCrawl, giving low resource language much better representation that mBERT. Tests on Spanish-English and tamil-English mixed text showed XLM-R beating mBERT consistently, especially when the language mixing was balanced and when the test data had lots of spelling variations. (Yandrapati Prakash Babu, 2021) The way XLM-R breaks into subword pieces using SentencePiece turns out to be helpful for Romanized text, since it can split unfamiliar spelling variants into meaningful smaller units (Supriya Chanda, 2024).

The gap: No one has systematically compared how mBERT and XLM-R perform on Romanized Nepali-English sentiment versus sequence models like BiLSTM trained from scratch on the actual data. It's unclear whether the transformers cross-language transfer abilities make up for the domain and script mismatch, or whether simpler model trained specifically on this data might work better for this low-resource situation. (Niraj Pahari, 2023)

2.3 Hyperparameter Tuning in Deep Learning

Model performance in deep learning systems depends on hyperparameter configuration, with sensitivity varying substantially across architecture. Research on BERT models has revealed that learning rate selection results the strongest influence on fine tuning success, with optimal values typically ranging between 1e-5 and 5e-5 for sentiment classification tasks. Batch size interacts with learning rate, with smaller batches requiring learning rate reduction to maintain training stability, while larger batches enable higher learning rates but may reduce generalization (Yihan Deng, 2025).

Within BiLSTM architecture, hidden layer dimension and dropout probability emerge as pivotal hyperparameters. Comparative studies contrasting manual adjustment against automated search methodologies establish that hidden dimensionality directly governs the model's capacity for encoding long range sequential dependencies, with sentiment classification benefitting from hidden dimensions spanning 128 to 512 units per directional component. Drop regularization proves indispensable for overfitting prevention on constrained datasets, with optimal dropout probabilities typically ranging 0.2 to 0.5 contingent on dataset scale and linguistic complexity (Gibson Nkhata, 2025).

Research Gap: Existing hyperparameters optimization research concentrate predominantly on monolingual English sentiment analysis or well-resourced code-mixed language pairs such as Hinglish. (Pranaydeep Singh, 2020) (Nikita Singhal, 2025) (Das, 2021) Publish literature lacks systematic investigation into optimal hyperparameter configurations tailored specifically for Romanized Nepali-English sentiment classification, where the distinctive combination of orthographic variability, code-mixing configuration, and resource scarcity may necessitate tuning strategies diverging from those effective for alternative language pairs. Additionally, comparative hyperparameter sensitivity analysis contrasting BiLSTM versus transformer architecture on identical Nepali-English materials is absent from current scholarship (Gibson Nkhata, 2025) (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

2.4 Identification Research Gaps

While sentiment analysis has become a SOTA research field and code-mixing is recognized as a real NLP challenge, several critical gaps remain:

Gap 1: Dataset Availability: There is few datasets for Romanized Nepali – English available, but there are in limited size and scope. Currently there is no large scale, robust and publicly available dataset, particularly from social media, despite social media being a major platform for Nepali language use generated content. Most of the Nepali Sentiment dataset focuses on Devanagari script and formal text, which does not reflect real-world social media writing practices.

Gap 2: Architectural Comparison: No one has systematically compared BiLSTM with Attention, mBERT and XLM-R specifically on Romanized Nepali-English sentiment data. While these architectures have been tested on Hinglish and Dravidian mixed pairs, linguistic difference between Nepali and those languages, plus Romanized scripts unique challenges, mean we can't just assume the finding transfer.

Gap 3: Hyperparameters Optimization: No published research has figured out optimal hyperparameter settings for deep learning models on Romanized Nepali-English sentiment. Since hyperparameter sensitivity varies by language, architecture, and dataset characteristics, we need empirically tested configurations specific to this language pari.

Why This Research Matters: These gaps collectively block development of effective sentiment analysis tools for Nepali's rapidly expanding social media environment. Content creators, businesses and researchers need dependable tools to understand public sentiment in the actual Romanized Nepali-English mixed language that dominates platforms like YouTube. Without systematic architectural comparison and hyperparameter optimization for this specific linguistic context, practitioners have no guidance for building accurate sentiment classifiers. And without cross-language consistency analysis, deployed models might behave unpredictable or show systematic biases when handling the natural language variation typical of bilingual Nepali Speakers.

2.5 Synthesis Matrix

Study	Language Pair	Platform/Domain	Script Type	Models Used	Dataset Size	Accuracy
(Pranaydeep Singh, 2020)	Hindi – English	SemEval 2020 Twitter	Romanised	Cross-Lingual Embeddings	15176	55.6%
(Anita Saroj, 2020)	Tamil-English Malayalam-English	YouTube Comment and HASOC	Native Scripts	BESRT-base	Over 1000 mentioned	59% for Tamil – English 6% for Malayalam -English
(Nikita Singhal, 2025)	Hindi – English	Facebook & HASOC	Romanised	XLM-R + CNN+BiLSTM (Hybrid)	Facebook = 12000 HASOC = 5000	73% for Facebook Dataset 82% for HASOC Dataset
(Das, 2021)	Hindi – English	Twitter	Romanised	BiLSTM + Attention, BERT	149207	BiLSTM + Attention = 73.25 BERT = 72.21
(Anuj Kumar, 2022)	Hindi-English Tamil - English	Social media	Mixed Language	XLM-R FastFormer	20000	72.20 %
(Niraj Pahari, 2023)	Nepali English	YouTube	Romanized	N/A (sociolinguistic study)	10702	mBERT = 68% XLM-R = 66%
My Research	Nepali – English	YouTube Comments	Romanized	BiLSTM + Attention, mBERT, XLM-R	10702	BiLSTM + Attention = 68% mBERT = 71% XLM-R = 72%

Table 1 Synthesis Matrix

3. Theoretical Foundation

3.1 Conceptual Foundation:

3.1.1 Sentiment Analysis

Sentiment analysis represents a computational approach within natural language processing aimed at extracting tone or subjective opinion embedded in textual content. At its core, this task involves classification: assigning text samples to predefined sentiment categories based on the affective stance expressed by the author (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

Most researchers use three main categories to cover the range of emotions people express online. Positive sentiment shows up when someone writes something favourable or enthusiastic, like when a YouTube commenter says " Yo movie ekdam ramro theyo!"(This move was good!). (Niraj Pahari, 2023). Negative sentiment fits factual statements or unclear expressions where you can't really tell if the person feels good or bad about something comments like "Video upload vayo ki nai?" (Has the video been uploaded?) fall into this category (Supriya Chanda, 2024).

Getting sentiment detection right gets complicated by several real challenges, especially with informal online writing. Sarcasm flips meaning on its head when someone writes "Great, another power cut," they're annoyed even though "great" usually means something positive. Negation also reverse sentiment "not happy" become negative even though "happy" by itself is positive, so models need to understand how words work together instead of just looking at them one by one. Models trained on one type of writing, like product review, often struggle when they encounter different style in political discussion or entertainment content, where people express emotions differently. But the biggest headache for social media sentiment analysis is informal language YouTube users write with slang, emojis, creative shortcuts, unusual punctuation, and both intentional and accidental spelling mistakes that confuse systems built for proper, edited text (Subalalitha Chinnaudayar Navaneethakrishnan, 2023) (Niraj Pahari, 2023).

3.1.2 Code-Switching and Romanization

Code-Switching is what happens when bilingual people naturally switch between two or more languages in the same conversation or even the same sentence. This isn't random multilingual speaker do it to express subtle meanings, show their identity, emphasize points, or match how the person they are talking to speaks (Supriya Chanda, 2024).

In today's Nepal, English has become a high-status language linked to education, technology, and global connection, especially among young people in cities (Niraj Pahari, 2023). So Nepali speakers routinely mix Nepali and English when they are chatting on social media, creating genuine mixed-language conversations. A typical YouTube comment might look like. "Aaja ko weather kati ramro cha yaar, I absolutely love it" (Today's weather is so nice friend, I absolutely love it!), where Nepali phrases and English words sit side by side in the same sentence1. (Niraj Pahari, 2023).

Romanized Nepali sometimes called Roman Nepali is when people write Nepali using English letters instead of the traditional Devanagari script. Unlike Devanagari, which has clear spelling rules, Romanized Nepali has no standard way to spell words, leading to wild variation. Different writers spell things based on how they think it sounds, what's easy to type on their keyboard, or how they pronounce words in their dialect, creating multiple versions of the same word:

Devanagari	Different Romanized Spellings
रमरो (good)	ramro, ramri, ramree, rāmro, ramroo, raami
बढी (more)	badi, badhi, badhee, badhii, badee
खाना (food)	khana, kana, khaana, khaan, kaana

Table 2 Example Romanised Nepali

This spelling chaos creates major problems for computer systems that expect words to be spelled consistently. A sentiment model trained to recognize "ramro" as indicating positive feelings might completely miss "ramri" or "ramree", treating them as unknown words and losing important emotional information (Supriya Chanda, 2024).

3.1.3 Challenges in Code-Switched Romanized Nepali-English Sentiment Analysis

When code-switching meets Romanization, sentiment analysis becomes uniquely difficult. Spelling inconsistency means the same Nepali word appears in countless variations across different people's writing sometimes even within the same person's comments making it impossible to match words reliably. Unknown vocabulary explosion happens because word embeddings and tokenizers built from formal text collections have never seen most Romanized Nepali words, leaving the model unable to understand these critical emotion carrying words.

Mixed writing system complexity emerges when standardized English sits alongside highly variable Romanized Nepali, forcing models to deal with two completely different spelling systems with different consistency levels at the same time. Data shortage remains severe while huge, labelled dataset exist for English sentiment and some resources cover Devanagari Nepali, there's essentially nothing for Romanized Nepali-English mixed text. Moreover, informal style makes things worse YouTube commenters use shortcuts ("bcz" instead of because), creative emoticons, deliberate misspellings for emphaisis ("soo good"), and platform-specific writing habits that stray even further form the written language most NLP systems were trained on (Sadia Alam, n.d.).

3.2 Technical Foundation

3.2.1 Generic Sentiment Analysis Pipeline

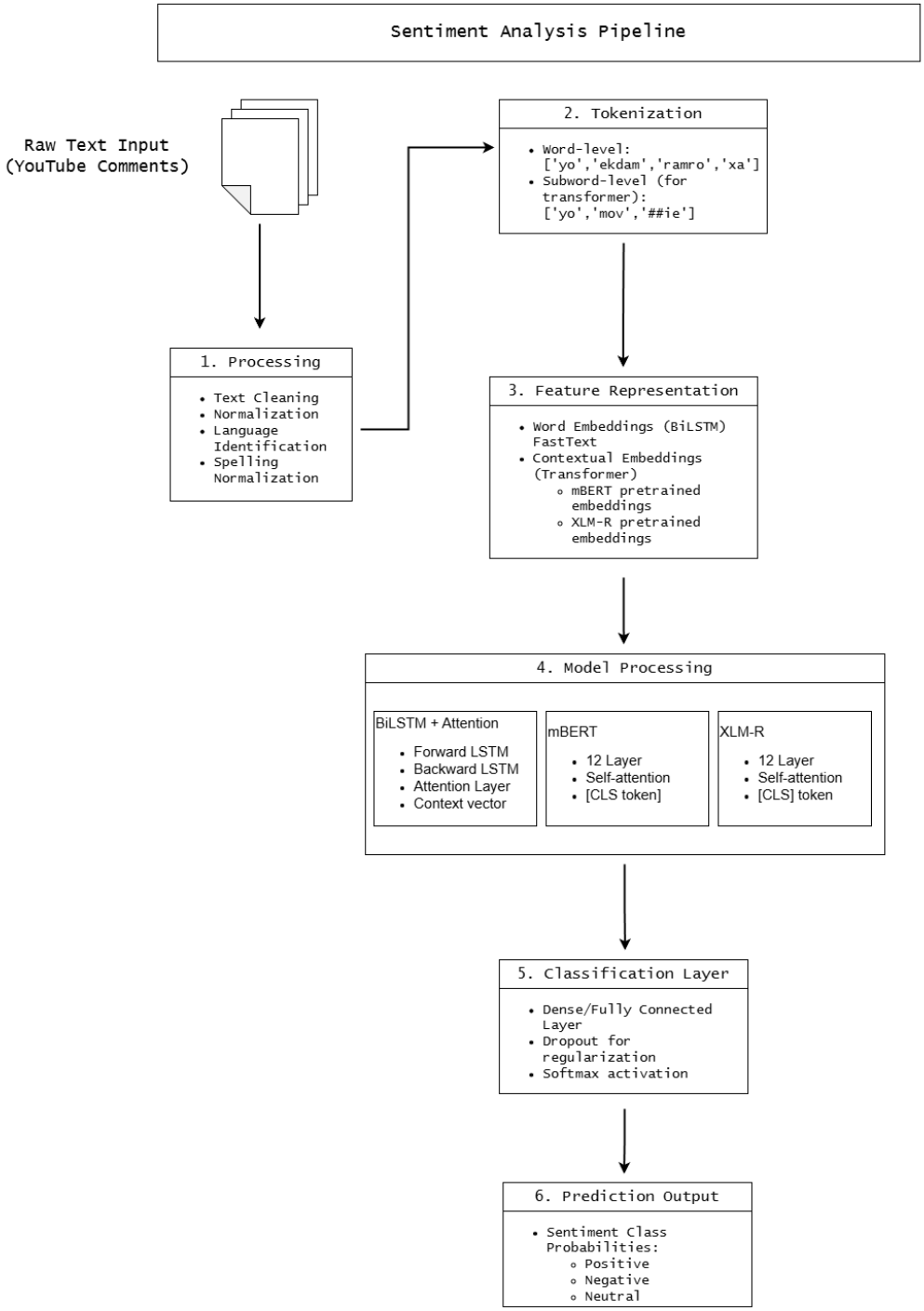


Figure 1 Sentiment Analysis Pipelines

3.2.2 Bidirectional Long Short-Term Memory (BiLSTM)

Long-Short Term Memory networks are specialized type of recurrent neural network built to capture patterns in sequence and remember things over long distances in the data. Unlike basic RNNs, which have trouble learning from long sequence because of vanishing gradients, LSTMs use gate mechanism such as, input gate, forget gate and output gat, that control information flow, letting the network selectively keep or discard information as it reads through the sequence (Sunil Gundapu, 2020).

BiLSTM takes this further by reading input sequence in both directions at once. A forward LSTM reads from start to finish, building up information about what came before each word. A backward LSTM reads from finish to start, gathering information about what comes after each word. The results from both directions get combined at each position, creating a complete picture that includes both left and right context (Arora, 2025).

For example, when analysing the sentence “I am feeling very good today because the weather is nice” one directional LSTM process from “I” to “nice” while the other processes from “nice” to “I”. The final representation for the word “good” would contain information about both the preceding words (“I am feeling very”) and the following words (“today because the weather is nice”), giving BiLSTM a more complete understanding of the sentence.

Attention Mechanism: Finding What Matters Most

BiLSTM understands context from both sides, but it treats all words equally when predicting. But in reality, some words are more important than others for sentiment analysis. For figuring out how someone feels, the words "terrible" and "amazing" in the sentence "The service was terrible, but the food was absolutely wonderful" are much more important than "the" or "but" (Arora, 2025).

The attention mechanism fixes this by showing the model which words are most important. It's like highlighting the most important parts of a text. Attention automatically figures out which words need more attention and gives them more weight. Attention doesn't assign all words the same weight. Instead, it assigns each word a score based

on how important it is to the prediction of the sentiment. Then, it highlights important words and downplays less important ones (Arora, 2025) (Gibson Nkhata, 2025).

BiLSTM with Attention: The Combined Approach

BiLSTM with Attention combines the strengths of both methods: BiLSTM takes context from both sides, and attention finds the most important words. This combination works especially well for sentiment analysis on code-mixed text, where words that show how someone feels can be in either language and can be anywhere in the sentence (Sunil Gundapu, 2020) (Arora, 2025).

A common BiLSTM with Attention architecture for sentiment classification works like this:

1. The embedding layer changes input tokens (words or parts of words) into number vectors. It can do this by starting with random values or using pre-trained embeddings.
2. The embedded sequence is processed in both directions by the BiLSTM layer(s), which makes hidden states that are aware of the context at each position (Naeem, n.d.).
3. The attention layer figures out how important each word position is and which words should have a bigger effect on the final prediction (Arora, 2025).
4. Weighted pooling uses attention weights to combine information from the whole sequence, which means that important words have a bigger effect on the final representation (Arora, 2025).
5. The classification layer uses fully connected layers to turn this weighted representation into sentiment class probabilities (Gibson Nkhata, 2025).

3.2.3 Transformer-Based Models: mBERT and XLM-RoBERTa

BERT (Bidirectional Encoder Representations from Transformers) changed NLP in a big way by using transformer architecture to do pre-training and then fine-tuning. Transformers use self-attention to figure out how all word pairs are related at once. This lets them process data in parallel and pick up long-distance connections quickly. BERT's bidirectional pre-training, which uses masked language prediction and sentence prediction on huge amounts of text, makes rich contextual representations that work well for specific tasks (Nusrat Jahan Prottasha, 2022).

Multilingual BERT (mBERT) goes a step further by pre-training one model on Wikipedia text from 104 languages, such as English and Nepali (written in Devanagari). It's amazing that mBERT can make connections between language representations even though it doesn't teach languages directly. This lets knowledge move between languages easily. A model optimised for sentiment analysis in English can unexpectedly categorise sentiment in Devanagari Nepali, indicating that mBERT has acquired language-agnostic semantic representations. mBERT saw Nepali written in Devanagari script, but its way of breaking words into smaller parts might work for Romanised versions as well because the word structure and sound correspondence are similar (Dou Hu, 2023).

XLM-RoBERTa (XLM-R) is a multilingual transformer that has been trained on 100 languages using 2.5 terabytes of CommonCrawl web data. This data is much more diverse and larger than the data used to train mBERT, which was only Wikipedia. XLM-R's training only uses masked language modelling (no sentence prediction), longer sequences, and dynamic masking. This is based on what was learnt from RoBERTa's work on English. XLM-R consistently beats mBERT on cross-language tasks, especially for low-resource languages that are better represented in CommonCrawl than in Wikipedia (Yandrapati Prakash Babu, 2021).

mBERT and XLM-R work through a fine-tuning process to classify feelings (Sunil Gundapu, 2020):

1. The pre-trained transformer works as a feature extractor by passing input through several self-attention layers.
2. A classification layer that is specific to the task is added on top of the encoder's [CLS] token representation. This layer is usually a dense layer with softmax.
3. The entire model, or just the classification layer, is trained on labelled sentiment data with much lower learning rates than when starting from scratch. This keeps the knowledge that was learnt before while adapting to the specific task.

Pre-trained transformers have a lot of benefits. For example, they have access to a lot of language knowledge that was encoded when they were pre-trained on billions of words. They also handle unknown and rare words better by using subword tokenisation. They also have the potential for cross-language transfer, where knowledge from high-resource languages helps low-resource targets. Finally, they get state-of-the-art results on many NLP benchmarks (Nusrat Jahan Prottasha, 2022).

But there are also problems: these models need a lot of computing power because they have hundreds of millions of parameters that need a lot of GPU memory; fine-tuning is hard because you have to choose the right learning rate for each layer; even though you pre-trained the model, there is a risk of overfitting on small datasets; and when you use models that were pre-trained on formal text (like Wikipedia or news), there is a risk of domain mismatch when you use them on informal social media language (Aman Priuanshu, 2021).

3.2.4 Hyperparameters and Model Training

Hyperparameter settings are the external controls that guide the learning process, rather than the information learnt from data. These settings have a big effect on how well models work. The learning rate tells the model how much to change its weights each time. If it's too high, the training becomes unstable and the loss goes up. If it's too low, the development is too slow or the model gets stuck. The learning rates for BiLSTM models are usually between 1e-3 and 5e-3. On the other hand, transformer fine-tuning needs much lower rates, between 2e-5 and 5e-5, to keep the knowledge that was already learnt from being lost (Gibson Nkhata, 2025).

The batch size tells the computer how many training samples to look at before it changes the weights. Smaller batches (16–32) make gradient estimates that are less stable and need lower learning rates to stay stable. But this noise could actually help with generalisation. Bigger batches (64–128) help you learn and train faster, but they don't work as well for generalising and need more GPU RAM (Yihan Deng, 2025).

Training epochs tell you how many times the training data goes through the whole thing. The model won't learn the patterns that are there if there aren't enough epochs. The model will remember the training details instead of learning general patterns if there are too many (Gibson Nkhata, 2025).

The hidden size (for BiLSTM) or embedding size (for transformers) affects how complex the patterns the model can show are. For BiLSTM sentiment work, hidden sizes of 128 to 512 units per direction are usually good. Larger sizes are needed if the lines between classes are more complicated (Sunil Gundapu, 2020).

The dropout rate encourages regularisation by randomly shutting off some neurones while training. This stops them from getting too used to certain patterns. For smaller datasets where overfitting is more likely to happen, dropout rates between 0.1 and 0.5 are good for reducing overfitting (Gibson Nkhata, 2025).

Weight decay uses L2 regularisation to punish large weights and make models simpler and more general. They are usually between 0.01 and 0.001.

You need to know about two ways that things can go wrong in order to understand how training works. When a model is underfitting, it doesn't have enough capacity or training, and both training and validation loss stay high and go down slowly. This means that the model hasn't even learnt the most basic patterns. It overfits when the model is too complicated or trained for too long. A big drop in training loss and a flat or rising validation loss show this. This means that the model has learnt things from training that don't apply to other situations. A good match is when the training and validation loss both go down steadily and stay close to each other (Gibson Nkhata, 2025).

Loss curves, which show how the training and validation losses change over time, are useful for diagnosing problems. The curves separate when the training loss goes down and the validation loss goes up. This means that the model is overfitting, which means that it needs more regularisation, a simpler model, or to stop training early. When both curves stay flat and high, it means that the model needs more capacity, longer training, or more features. This is called underfitting. The fact that both curves are going down smoothly together shows that the model is learning well because it can use what it learnt during training to validation (Yihan Deng, 2025).

This theoretical framework sets up the intellectual and technological background needed to understand why certain architectural choices were made, how hyperparameter tuning was done, and what assessment methods were used for this research. The particular difficulties associated with code-switched Romanised Nepali-English sentiment analysis clarify the justification for a thorough comparison of BiLSTM and transformer methodologies, while the technical characteristics of each architecture inform the investigation of hyperparameter ranges and the expected performance trade-offs in the following chapters (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

4. Research Methodology

4.1 Research Design

This is an **experimental, comparison study** with qualitative analysis. The overall design involves:

- **Data Preparation:** The dataset used in this study was obtained from (Pahari, 2023), who collected and annotated Romanized Nepali and English code-mixed YouTube comments for his research.
- **Model Development:** Implementing three model architecture.
    - BiLSTM with Attention Mechanism
    - mBERT
    - XLM-R
- **Hyperparameter**: Systematically exploring configuration for each model.
- **Evaluation**: Assessing performance on a held-out test set using standard metrics.

4.2 Dataset

4.2.1 Data Source and Collection

This research utilizes the Romanized Nepali – English sentiment analysis dataset developed by (Pahari, 2023). The author collected comments from YouTube, top 10 YouTube channels in Nepal of news & political category, manually annotating each for sentiment polarity. (Pahari, 2023) Author collected all the comments and threads of top 50 videos of each channels using YouTube API. The author skipped the comments containing Devanagari scripts. The author uses annotators to annotate each comment into three sentiment classes: Positive, Neutral and Negative. The annotator annotated 10,702 comments in total (Pahari, 2023).

4.2.2 Data Statistics

Metrics	Value
Total Instances	*10702*
Positive Instances	*3460*
Negative Instances	*3648*
Neutral Instances	*3594*

Table 3 Dataset Statistics

Example Instances: 1. "*very very nice voice kamala nanu*" [Neutral] 2. "*derai ramro kam sir*" [Positive]

4.2.3 Train/Validation/Test

The dataset was split as follows:

- Training set: 80% (8561 instances) used to train models.
- Validation set 10% (1070 instances) used for hyperparameter tuning and early stopping.
- Test set 10% (1071 instances) held out for final evaluation, never seen during training or tuning.

4.3 Preprocessing

4.3.1 Text cleaning

While data collection author priorities only the comment and threads, he didn't collect any URLs or any mentions. The normal text cleaning like removing extra whitespaces, were done by the author. Mostly the author processed the raw text without changes to get most out of the dataset in his study, (Niraj Pahari, 2023) there were some emoji mention in the dataset, but for this study emoji were ignored, alternatively it can be converted to textual description.

4.3.2 Tokenization

For BiLSTM: uses custom regex-based tokenizer specifically designed for code-mixed text. It converts all characters to lowercase to ensure consistency. This treats “Movie”, “MOVIE”, and “movie” as the same token, which is crucial. And it

- reduces the vocabulary size
- prevents duplicate embeddings for same words with different cases
- improves model generalization

For mBERT: uses sub-word pre-trained tokenizers from Hugging Face transformer library, which handles code-switching through contextual understanding. mBERT uses WordPiece with 110k vocabulary (Anon., n.d.).

For XLM-R: also uses sub-word pre-trained tokenizers from Hugging Face transformer library, which handles code-switching through contextual understanding. XLM-R uses SentencePiece with 250k vocabulary (Anon., 2020).

4.4 Model Development

4.4.1 BiLSTM Architecture

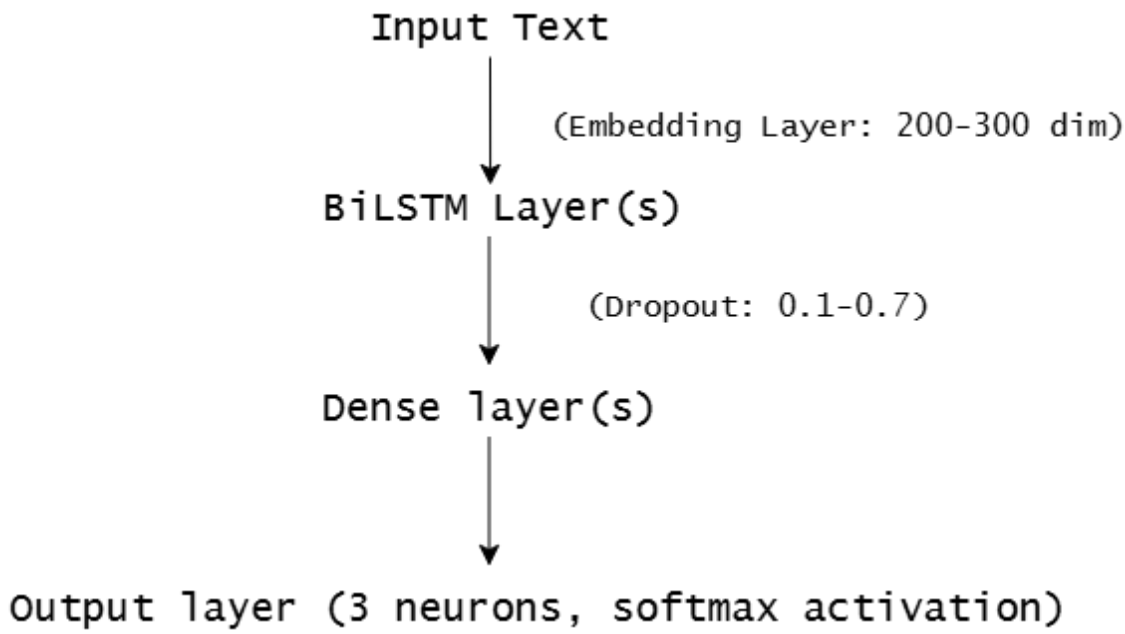


Figure 2 BiLSTM Architecture

Key Components:

- Embedding Layer: Initializes word vectors randomly or from per-trained embeddings.
- BiLSTM:
  - Hyperparameter: hidden_dim = {128, 256, 512}
  - Num_layer: 2
  - Dropout:
    - Hyperparameter: dropout_rate = {0.3-0.7}

4.4.2 mBERT Architecture

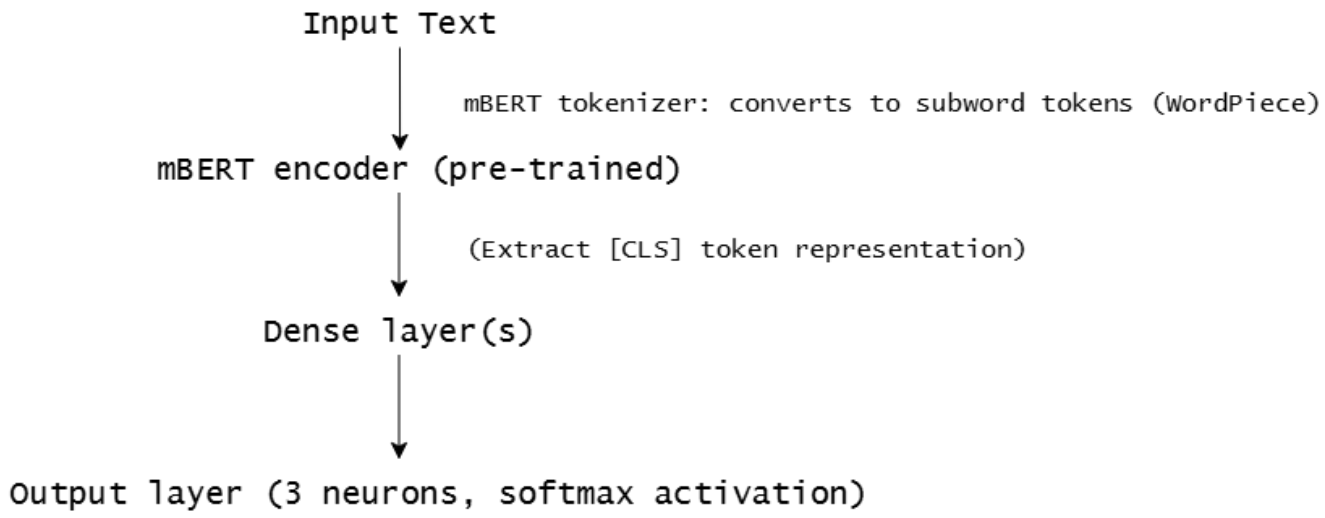


Figure 3 mBERT Architecture

4.4.3 XLM-R Architecture

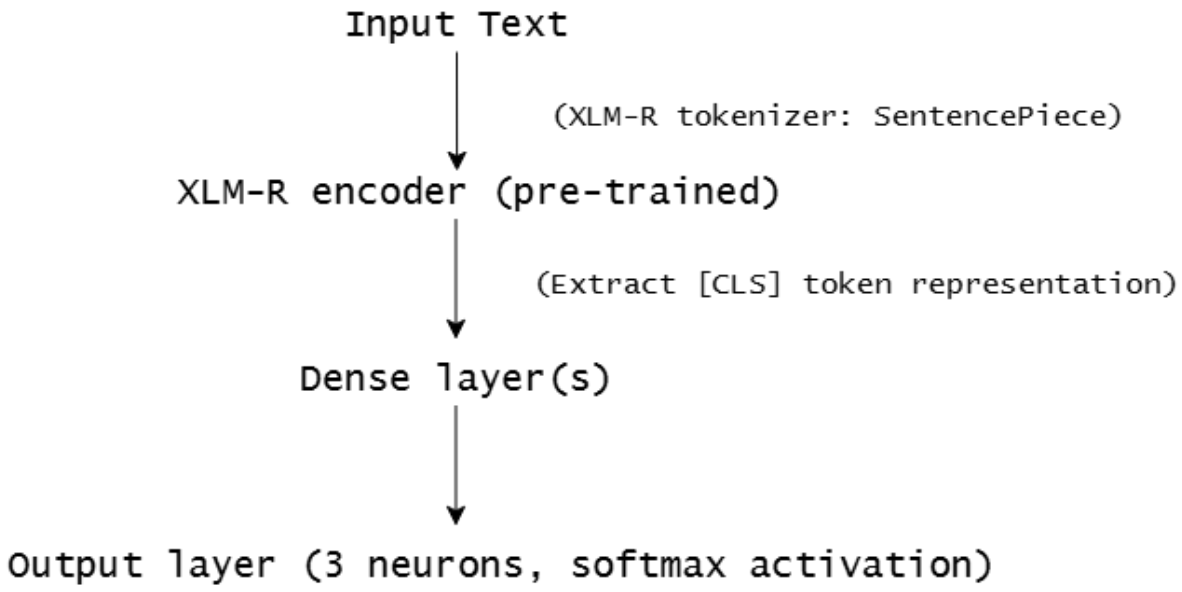


Figure 4 XLM-R Architectures

4.5 Hyperparameter Tuning Strategy

A manual grid search was conducted over the following hyperparameter space:

Model	Hyperparameter	Values
BilSTM	Learning rate	1e-3
	Batch Size	32
	Dropout	0.3,0.5, 0.6, 0.7
	Hidden_dim	256,128,
	Epoch	20, 6, 3
	Weight_decay	1e-4
mBERT	Max_length	192, 256
	Batch size	16, 32
	Lr BERT encoder	1e-5, 5e-6, 8e-6, 1.5e-5
	Lr Classifier	5e-4, 2e-4, 3e-4, 4e-4
	Weight decay	0.01, 0.02, 0.015
	Epsilon	1e-8
	Dropout rate	0.1, 0.3, 0.2, 0.15
	Epoch	10, 5
XLM-R	Max length	128, 256
	Batch size	16, 32
	Learning rate (encoder)	2e-5, 3e-5
	Learning rate (classifier)	1e-4, 2e-4
	Weight decay	0.01, 0.05
	Epoch	5, 7
	Warm up ratio	0.1, 0.15
	Dropout rate	0.3, 0.4

Table 4 Hyperparameter Tuning Strategy

Each hyperparameter combination was trained and evaluated on the validation set. The configuration with the highest weighted F1score on the validation set was selected as the “optimal” configuration for that model.

4.6 Evaluation Metrics and Strategy

4.6.1 Standard Metrics

Models are evaluated on the test set using:

- Accuracy: (TP + TN) / (TP + TN + FP + FN)
- Precision (per class): TP / (TP + FN)
- Recall (per class): TP / (TP + FN)
- F1-score (per class): 2 × (Precision × Recall) / (Precision + Recall)
- Macro-averaged F1: Average of per-class F1 scores (treats all classes equally).
- Weighted F1: Weighted average based on class frequency (accounts for imbalance).

Primary metric: Weighted F1-score (since classes are balanced, macro F1 is also reported).

4.6.2 RQ1 Evaluation

To answer RQ1 (which model will perform best on code-mixed Romanized Nepali and English text)

1. Train each model variant (all hyperparameter combination)
2. Record validation F1 during training; plot training loss vs validation loss curves
3. Select the configuration with highest validation F1.
4. Compare test set performance across models and configurations.

4.6.3 RQ2 Evaluation: Cross-Lingual Consistency

To answer RQ2 (how well does the best model generalize to equivalent sentences in different language forms)

Paired-sentence evaluation:

1. Manually construct a set of semantically equivalent sentences pairs.
    a. Pair 1: Romanized Nepali version
    b. Pair 2: Code-mixed version (all same sentiment)
2. For the best-performance model, run inference on each sentence and record:
    a. Predicted sentiment class.
    b. Softmax probabilities for each class.
    c. Whether the prediction matches the gold label (correct/incorrect)
3. Compute aggregate statistics:
    a. Accuracy per language variant: Accuracy for Romanized Nepali and code-switched separately.
    b. Consistency rate: % of pairs where all three variants are predicted with the same label.
    c. Confidence metrics: Mean softmax probability for correct predictions spread across variants.
4. Analyse results:
    a. Is there systematic bias toward English?
    b. Does codeswitch help or hurt?
    c. Where does the model struggle?

1. Implementation, Results, and Discussion

5.1 Experimental Setup

Hardware and Software:

- GPU: NVIDIA [RTX 4070] with 8 GB VRAM
- RAM: 32 GB
- Framework: PyTorch, Hugging Face Transformer, scikit-learn, Matplotlib/Seaborn
- Implementation language: Python 3.12.7
- Tools: Jupyter Notebook

5.2 Implementation Details

BiLSTM:

```
# Simplified architecture (PyTorch)
class BiLSTMAttentionSentiment(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim,
                 num_layers, dropout, num_classes=3):
        super().__init__()
        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embedding_dim,
                                      padding_idx=0)

        # Bidirectional LSTM
        self.bilstm = nn.LSTM(embedding_dim, hidden_dim,
num_layers,
                              batch_first=True,
bidirectional=True,
                              dropout=dropout if num_layers > 1
else 0)

        # Attention mechanism
        self.attention = nn.Linear(2 * hidden_dim, 1)

        # Classification head
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(2 * hidden_dim, num_classes)

    def forward(self, x, lengths):
```

```
        # Embedding: (batch, seq_len) → (batch, seq_len,
embed_dim)
        embedded = self.embedding(x)

        # Pack padded sequences for efficiency
        packed = pack_padded_sequence(embedded, lengths,
                                      batch_first=True,
                                      enforce_sorted=False)

        # BiLSTM: (batch, seq_len, 2*hidden_dim)
        lstm_out, (h_n, c_n) = self.bilstm(packed)
        lstm_out, _ = pad_packed_sequence(lstm_out,
batch_first=True)

        # Attention mechanism
        attention_scores = self.attention(lstm_out)  # (batch,
seq_len, 1)
        attention_weights = F.softmax(attention_scores, dim=1)

        # Weighted sum using attention
        context_vector = torch.sum(attention_weights * lstm_out,
dim=1)

        # Classification
        output = self.dropout(context_vector)
        logits = self.fc(output)

        return logits, attention_weights
```

Hyperparameter Search Space:

Hyperparameter	Selection Range
Batch_size	32
Dropout	0.3,0.5,0.6, 0.7
Hidden Dimension	128, 256
Learning Rate	1e-3
Epoch	3, 6, 20
Weight Decay	1e-4

Table 5 Hyperparameter Search Space 1

Multilingual BERT (BERT):

```
# Using Hugging Face Transformers
from transformers import BertForSequenceClassification,
BertTokenizer

# Load pre-trained mBERT
model = BertForSequenceClassification.from_pretrained(
    'bert-base-multilingual-cased',
    num_labels=3,
    hidden_dropout_prob=0.1,
    attention_probs_dropout_prob=0.1
)

tokenizer = BertTokenizer.from_pretrained(
    'bert-base-multilingual-cased'
)

# Fine-tuning configuration
# Option 1: Fine-tune all layers
for param in model.parameters():
    param.requires_grad = True

# Option 2: Freeze lower layers (if needed)
# for param in model.bert.embeddings.parameters():
#     param.requires_grad = False
```

Hyperparameter Search Space:

Hyperparameter	Selection Range
Max length	192
Batch size	16, 32
Learning rate (encoder)	1e-5, 5e-6, 8e-6
Learning rate (classifier)	5e-4, 2e-4, 3e-4
Weight decay	0.01, 0.02, 0.015
Epoch	5,10
Warm up ratio	0.05, 0.10
Dropout rate	0.1, 0.2, 0.3

Table 6 Hyperparameter Search Space 2

XLM-RoBERTa (XLM-R):

```
from transformers import XLMRobertaForSequenceClassification,
XLMRobertaTokenizer

# Load pre-trained XLM-R
model = XLMRobertaForSequenceClassification.from_pretrained(
    'xlm-roberta-base',
    num_labels=3,
    hidden_dropout_prob=0.1,
    attention_probs_dropout_prob=0.1
)

tokenizer = XLMRobertaTokenizer.from_pretrained('xlm-roberta-
base')
```

Hyperparameter Search Space:

Hyperparameter	Selection Range
Max length	128, 256
Batch size	16, 32
Learning rate (encoder)	2e-5, 3e-5
Learning rate (classifier)	1e-4, 2e-4
Weight decay	0.01, 0.05
Epoch	5, 7
Warm up ratio	0.1, 0.15
Dropout rate	0.3, 0.4

Table 7 Hyperparameter Search Space 3

5.2.1 Training Procedure

For each model

1. Initialize Model and move to GPU
2. Create train/val/test dataloaders
3. Set up optimizer and loss function
4. For each epoch:
    a. Train on training set
    b. Evaluate on validation set, record loss and F1
    c. If f1 validation F1 improves save model
5. After training load best model and evaluate on test set.

5.3 Model Performance on the Main Dataset (RQ1)

5.3.1 Hyperparameter Tuning Results

B**iLSTM**

Batch size	Dropout	Hidden dim	Learning rate	Epoch	Weight decay	Val F1
32	0.3	256	1e-3	20		0.67
32	0.5	128	1e-3	6	1e-4	0.68
32	0.6	128	1e-3	3	1e-4	0.67
32	0.7	128	1e-3	3	1e-4	0.68

Table 8 Hyperparameter Tuning (BiLSTM

Best BiLSTM configuration: hiddendim = 128, dropout = 0.7, learning rate = 1e-3

mBERT

Max length	Batch Size	Learning rate (encoder)	Learning rate (classifier)	Weight_decay	Epoch	Warm up ratio	Dropout rate	Val F1
192	16	1e-5	5e-5	0.01	10	0.05	0.1	0.72
192	32	5e-6	2e-4	0.02	10	0.10	0.3	0.69
129	32	8e-6	3e-4	0.015	5	0.10	0.2	0.69
256	16	1.5e-5	4e-5	0.01	10	0.06	0.15	0.71

Table 9 Hyperparameter Tuning (mBERT)

Best mBERT configuration: max length = 256, batch size = 16, lr encoder = 1.5e-5, lr classifier = 4e-5, weight decay = 0.01, epoch = 10, warm-up = 0.6, dropout rate = 0.15, val f1 = 0.71

XLM-R

Max length	Batch Size	Learning rate (encoder)	Learning rate (classifier)	Weight_decay	Epoch	Warm up ratio	Dropout rate	Val F1
128	16	2e-5	1e-4	0.01	5	0.1	0.3	0.75
256	32	3e-5	2e-4	0.05	7	0.15	0.4	0.74
128	32	2e-5	1e-4	0.04	3	0.1	0.4	0.72

Table 10 Hyperparameter Tuning (XLM-R)

Best XLM-R configuration: max length = 128, batch size = 16, lr encoder = 2e-5, lr classifier = 1e-4, weight decay = 0.01, epoch = 5, warm up ratio = 0.1, dropout rate = 0.3

5.3.2 Test Set Performance

It presents final test set performance for the optimal configuration for each model.

Model	Accuracy	Macro F1	Weighted F1	Pos F1	Neg F1	Neu F1
BiLSTM	0.68	0.69	0.68	0.77	0.68	0.61
mBERT	0.71	0.72	0.71	0.79	0.71	0.65
XLM-R	0.75	0.75	0.75	0.82	0.76	0.67

Table 11 Optimal Model Test Performance

Key Findings: XLM-R achieved the best overall performance (75% weighted F1), mBERT achieved 71% weighted F1, outperforming BiLSTM – BiLSTM achieved 69% weighted F1, showing its relative weakness on this task.

5.3.3 Training Curves and Loss Analysis

Here, this chapter presents the training and validation loss curve for the optimal configuration of each model.

Figure: Training vs Validation Loss Curve

BiLSTM:

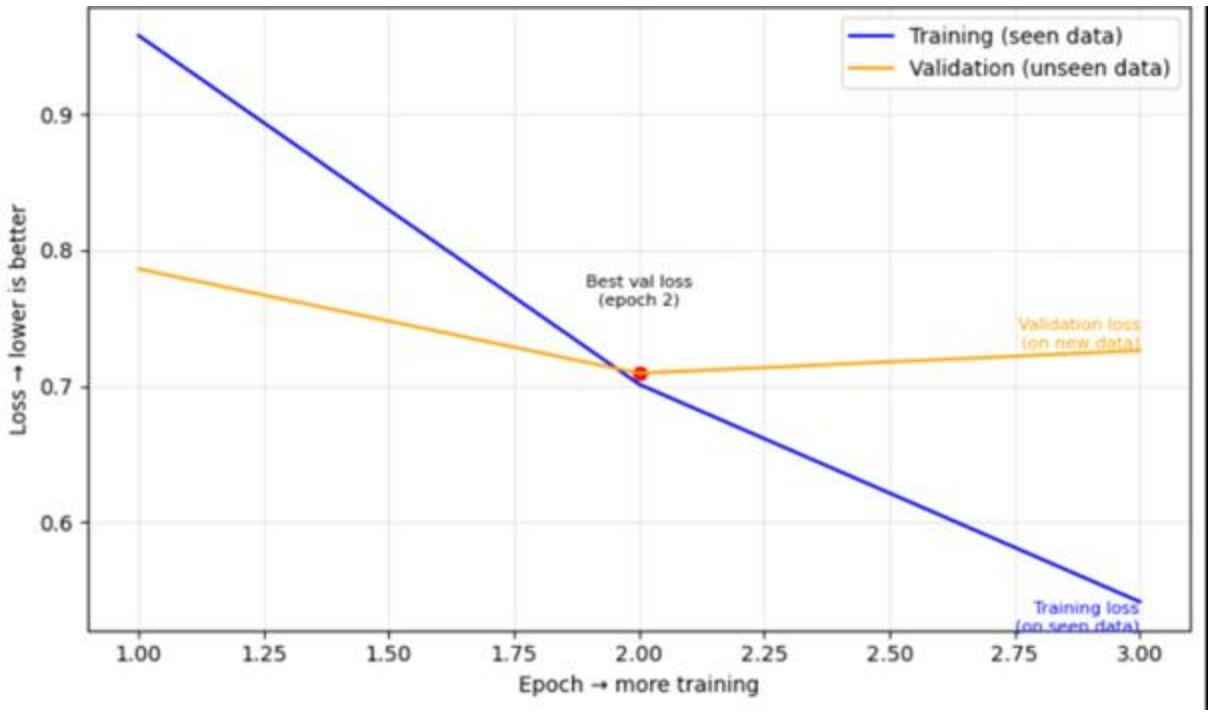


Figure 5 Training vs Validation Curve (BiLSTM + Attention)

mBERT:

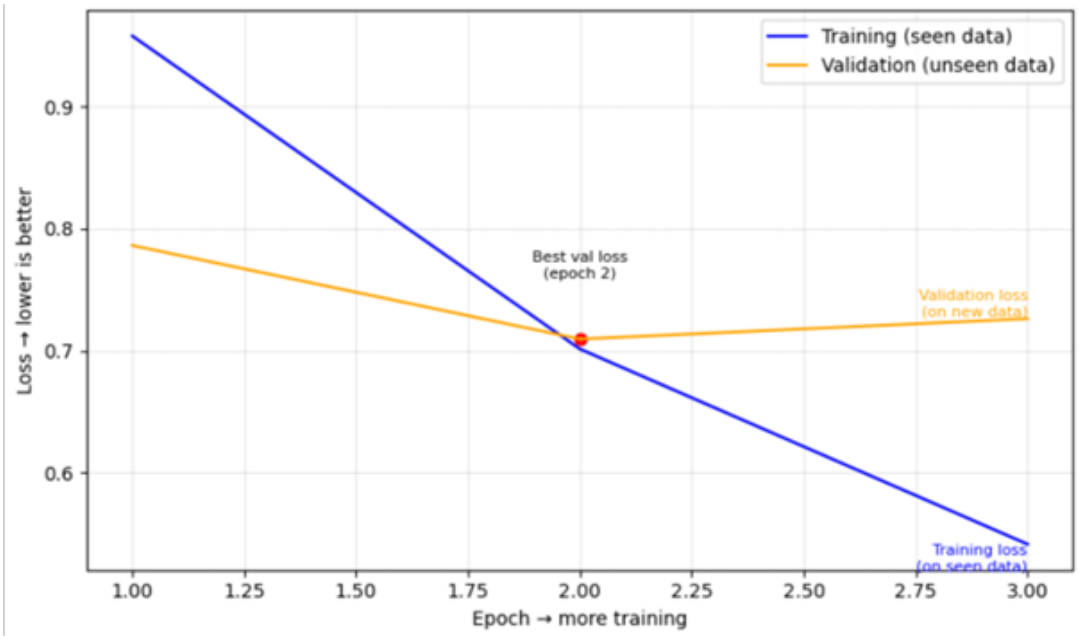


Figure 6 Trainig vs Validation Curve (mBERT)

XLM-R:

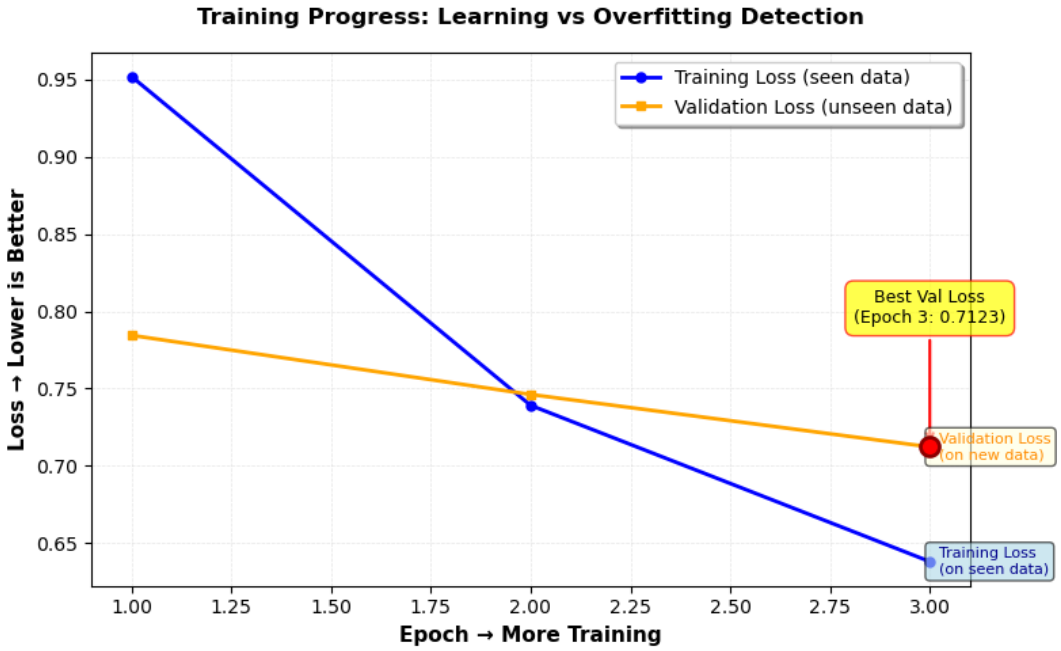


Figure 7 Trainig vs Validation Curve (XLM-R)

Observation:

The research demonstrates that all three models have their greatest validation performance at Epoch 2. However, XLM-RoBERTa generalises the best, with a modest train-validation loss gap (~0.19 by Epoch 3), while BiLSTM+Attention mildly overfits and mBERT severely overfits in later epochs. These results show that multilingual pre-trained transformers, especially XLM-RoBERTa, which has a bigger subword vocabulary and more multilingual pre-training, are better for Romanised Nepali-English sentiment analysis than a BiLSTM trained from scratch. However, early stopping should be used to avoid overfitting on the small dataset.

5.3.4 Discussion of RQ1 Results

Why does XLM-R outperform others?

1. Multilingual pre-training: XLM-r is trained on more language than mBERT, with a larger corpus. This makes it more knowledgeable about other languages.
2. Better handling of Romanised Nepali: Even though XLM-R was not specifically trained on Romanised Nepali. It has a huge multilingual vocabulary (250k tokens) that includes subword units that can break down Romanised terms that are new to you. For instance, “ramri” might be broken down into two parts: “ram” and “ri”. Both of these words are likely to be in the lexicon.
3. Advantage of subword tokenization: XLM-R’s SentencePiece tokenizer is better at handling spelling mistakes than BiLSTM’s word level tokenization.
4. Pre-trained representation: mBERT and XLM-R are better than BiLSTM because they use pre-trained contextual embeddings that capture subtle semantic meaning.

Why does BiLSTM not work as well as it should?

1. Not enough data: BiLSTM is trained from scratch on about 1,500 training examples, which is not a lot for neutral networks. Pre-training helps transformers.
2. Overfitting: The BiLSTM model is showing signs of overfitting (train-val divergence), which means that the model’s capacity is too high for the data it is trained on.

3. Lexicon Limitations: BiLSTM depends on a lexicon that comes from training data. When Romanised Nepali words are rare or spelt wrong, they become UNK>tokens, which means they lose information.

The best model for this problem is XLM-R, with a learning rate of 2e-5, a batch size of 32 , and 3 epoch.

5.4 Cross-Lingual Consistency Analysis (RQ2)

We made a test set of 11 sentence triplets that all mean the same thing. Each triplet has a Romanised Nepali version, an English version, and a code-switched version. All three variations have the same gold sentiment label.

Example Pairs:

1. Positive:
   - Romanized Nepali: “ yo movie ekdam ramrod cha.”
   - English: “this movie is very good.”
   - Code-Switched: “yo movie is ekdam ramrod”
2. Negative:
   - Romanized Nepali: “malai mannparena”
   - English: “i did not like it.”
   - Code-mixed: “malai, I did not like.”
3. Neutral:
   - Romanized Nepali: “yo thikai xa.”
   - English: “this is okay.”
   - Code-Switch: “yo okay cha”

5.4.1 Per-variant Accuracy and Confidence

This chapter present for the best model (XLM-R) on each language variant.

Variant	Accuracy	Avg. Confidence	Weighted F1	Pos F1	Neu F1	Neg F1
Romanised Nepali	0.60	0.86001333	0.5493	0.7692	0.3333	0.5455
English	0.8667	0.81821333	0.8611	1.000	0.8333	0.7500
Code-Switched	0.5333	0.84366	0.4815	0.7143	0.4444	0.2857

Table 12 Per-Variant Accuracy and Confidence Score

Findings:

- Accuracy differential: English variants achieved 86.67% accuracy, Romanized Nepali only 60%, a gap of 26 percentage points.
- Weighted F1: English variants had higher average confidence.
- Confidence Differential: The average confidence score shows how sure the model is about its predictions of emotion. As you can see, the model had the highest average confidence on Romanised Nepali text (0.86), then Code-Switched data (0.84), and finally English (0.82). This means that the model is most sure of itself when it looks at Romanised Nepali input, but it still does well with mixed-language and English datasets.

5.4.2 Consistency Rate

A pair is "consistent" if all three versions (Romanised Nepali, English, and code-switched) get the same projected label and that label matches the gold label.

Results:

- Full Consistence: 46.67%
- We looked at 15 different language pairs. To check for cross-lingual stability, we looked at how consistent the sentiment predictions were across the three language versions (Romanised Nepali, English, and Code-Switched).
- Results for Prediction Consistency
- The prediction was the same in all three languages: 46.67%
- All three forecasts were right when they were the same: 46.67%

These results show that in almost half of the examples that were looked at, the sentiment model made predictions that were completely consistent and right in all three language forms

.

5.4.3 Discussion of RQ2 Findings

The cross-lingual consistency experiment used 11 sets of three sentences, each with a Romanised Nepali version, an English version, and a code-switched version that all say the same thing (positive, negative, or neutral). For instance, "yo movie ekdam ramro cha," "this movie is very good," and "yo movie is ekdam ramro" make up one positive triplet, whereas "malai mannparena," "I did not like it," and "malai, I did not like" make up a negative triplet with the same gold labels. This configuration lets you directly examine if the model processes semantically similar content the same way in different languages and code-mixing patterns.

The best model (XLM-R) shows a significant bias towards English in the per-variant results. The accuracy of English sentences was 86.67%, and the weighted F1 score was 0.8611. The class-wise F1 values were 1.0 for positive, 0.8333 for neutral, and 0.75 for negative. Romanised Nepali, on the other hand, only got 60% accuracy and a weighted F1 of 0.5493. It did substantially worse for neutral (0.3333) and negative (0.5455) inputs. Code-switched inputs did the poorest overall, with 53.33% accuracy and 0.4815 weighted F1. The 26-point difference in accuracy between English (86.67%) and Romanised Nepali (60%) shows that the model works far better on English text, even though the feelings behind it are the same. One possible reason is pre-training skew: XLM-R has seen a lot more well-formed English during pre-training than noisy Romanised Nepali. The English in the dataset follows standard spelling and grammar rules, while the Romanised Nepali has a lot of spelling variation and inconsistent Romanisation, which makes it harder to align subwords with pre-trained representations.

The average confidence offers a somewhat different story: the model is most sure on Romanised Nepali (0.86), then code-switched (0.84), and least confident on English (0.82). This is strange because English has the best actual accuracy. This means that in this multilingual situation, more confidence doesn't always guarantee superior accuracy. The model might be too sure of its Romanised Nepali predictions, even when they're erroneous, but it might be more accurate on English. The model "believes" its Romanised Nepali predictions firmly, yet it makes more mistakes in that language. On the other hand, its English predictions are more confident and more often true.

5.5 Error Analysis

5.5.1 Overview of Failure Cases

The XLM-R model only got 46.67% full consistency out of 15 triplets (45 total phrases). This means that 8 triplets (53.33%) had at least one misclassified variant. A close look at these mistake scenarios shows that there are systematic patterns linked to linguistic issues that make the model less reliable across languages, especially for Romanised Nepali and code-mixed inputs (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

5.5.2 Error Type 1: Negation Misinterpretation

Test Group 10: Complete Negation Failure:

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"timro kaam bilkul ramro bhayena"	Negative	Positive	86.96%
English	"your work did not turn out well"	Negative	Neutral	54.05%
Code-mixed	"timro work bilkul ramro bhayena"	Negative	Positive	97.31%

Table 13 Complete Negation Misinterpretation

Analysis: This triplet demonstrates disastrous negation handling in all variations. The Romanised Nepali and code-mixed versions have the negation marker "bhayena" (did not become), but the model predicts Positive with very high confidence (85.96% and 97.31%, respectively). This seems to be because it gives too much weight to the positive adjective "ramro" (good/well) and doesn't pay any attention to the negation. The English version does a little better because it doesn't make a positive forecast, but it still gets the classification wrong by calling it Neutral instead of Negative. This indicates that the model's attention mechanism inadequately integrates negation with sentiment-laden words in Romanised Nepali, perceiving "ramro" and "bhayena" as distinct semantic units instead of compositely reversing polarity.

Test Group 8: Partial Negation Failure:

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"yo geet sunna naramro lagyo"	Negative	Positive	87.46%
English	"this song sounds terrible"	Negative	Negative	62.13%
Code-mixed	"yo song sunna naramro lagyo"	Negative	Positive	99.55%

Table 14 Partial Negation Failure

Analysis: The word "naramro" (not good/bad) has an inherent negative ("na-" prefix + "ramro"), however both Romanised Nepali and code-mixed versions are confidently misclassified as Positive (87.46% and 99.55%). The English version with the clear negative word "terrible" is correctly categorised (though with a low level of confidence at 62.13%), which shows that the model is better at understanding English negation than Nepali morphological negation indicators. This pattern suggests that XLM-R's subword tokenisation might break "naramro" into ["na", "##ramro"] without understanding that the "na-" prefix changes the polarity of sentiment in Nepal.

5.5.3 Error Type 2: Neutral Class Confusion

Test Group 12" "Okay/Thikai" Ambiguity:

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"khana thikai cha"	Neutral	Negative	90.94%
English	"the food is okay"	Neutral	Neutral	61.29%
Code-mixed	"food thikai cha"	Neutral	Negative	73.59%

Table 15 Neutral Class Confusion

Analysis: The term "thikai" in Nepali means "okay" or "acceptable." It is meant to be neutral, however it is expected to be negative in both Romanised Nepali (90.94% confidence) and code-mixed (73.59% confidence) variants. It is correct to say that "okay" in English is Neutral. This means that "thikai" may not have been used enough in the training data or may have been used more often in unfavourable situations, which would have made the model acquire a false negative connection. The high confidence in the inaccurate forecast (90.94%) is especially worrisome since it shows that the person is too sure of themselves without really comprehending.

Test Group 13: "Milcha/Acceptable" Misclassification

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"yo geet sunna milcha"	Neutral	Negative	83.03%
English	"this song is acceptable"	Neutral	Neutral	81.10%
Code-mixed	"yo song sunna milcha"	Neutral	Negative	42.54%

Table 16 Misclassification

Analysis: "Milcha" (acceptable/can be) in pure Romanised Nepali is incorrectly rated as Negative with high confidence (83.03%), whereas both English and code-mixed versions are correctly classed as Neutral. This partial success in the code-mixed version (42.54% confidence, lower but correct) implies that the English word "song" improves the model by giving it more context, which backs up the earlier discovery that English tokens make predictions more accurate. But when the whole sentence is in Romanised Nepali, the model automatically gives a wrong negative meaning.

Test Group 14: Positive Drift for Neutral Sentiment

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"aaja ko din thikai cha"	Neutral	Positive	55.44%
English	"today is an average day"	Neutral	Neutral	79.36%
Code-mixed	"aaja ko day thikai cha"	Neutral	Positive	69.84%

Table 17 Positive Drift for Neutral Sentiment

Analysis: In this case, both Romanised Nepali and code-mixed versions lean towards Positive forecasts (55.44% and 69.84%) for a neutral statement that includes "thikai" (okay/average). The model seems to think that "din" (day) and "aaja" (today) together with "thikai" have a positive valence. This could be because words relating to days were used with good feelings in the training data (for example, "wonderful day"). The English phrase "average day" is accurately identified as Neutral with a greater level of confidence (79.36%). This shows that English inputs are better calibrated.

5.5.4 Error Type 3: Code-mixed Specific Failures

Test Group 7: Code-mixed Underperformance:

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"ma yo khana maanparena"	Negative	Negative	92.16 %
English	“I did not like this food”	Negative	Negative	96.26%
Code-mixed	“ma yo food mannparena”	Negative	Neutral	60.09%

Table 18 Code-mixed Underperformance

Analysis: Both monolingual versions (Romanised Nepali and English) properly identify this negative attitude with high confidence (92.16% and 96.26%), while the code-mixed version fails, predicting Neutral with only 60.09% confidence. The term "maanparena" (did not like) plainly has a negative meaning, and the pure Romanised Nepali sentence processes it correctly. However, when it is paired with the English word "food," the model's prediction gets worse. This indicates that code-mixing upsets the model's sentiment composition mechanisms; the sudden language switch may hinder the flow of contextual embeddings through the transformer layers, resulting in the negation in "maanparena" losing its efficacy.

Test Group 9: Code-Mixed Misinterpretation

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"yesto kharab din aaja”	Negative	Negative	99.47%
English	“such a terrible day today”	Negative	Neutral	47.50%
Code-mixed	“yesto terrible day aaja”	Negative	Neutral	63.97%

Table 19 Code-mixed Misinterpretation

Analysis: The Romanised Nepali version is accurately classified with a very high level of confidence (99.47%). However, both the English and code-mixed versions fail, predicting Neutral instead of Negative (47.50% and 63.97% confidence). This is strange because "terrible" is a clear negative word in English that should make it negative. The model is probably confused because of the way the sentences are put together. For example, "such a terrible day today" may look like neutral reporting patterns in the training data ("today is..."). The code-mixed version "yesto terrible day aaja" mixes Nepali framing words ("yesto"=such, "aaja"=today) with English "terrible" in a way that breaks up the negative signal across languages.

5.5.5 Error Type 4: Overconfident Misclassifications

Test Group 15: Low-Confidence Ambiguity

Variant	Sentence	True Label	Predicted	Confidence
Romanized Nepali	"timro kaam theek lagyo”	Neutral	Negative	43.30%
English	“your work seems fine”	Neutral	Neutral	78.43%
Code-mixed	“timro work theek lagyo”	Neutral	Positive	94.06%

Table 20 Overconfident Misclassification

Analysis: The Romanised Nepali prediction is erroneous, but it only has a 43.30% chance of being right. The probabilities show that Negative (43.30%) and Positive (41.99%) are almost equal, which shows that there is real uncertainty. But the code-mixed version is sure to be erroneous; it says Positive with 94.06% confidence when the genuine label is Neutral. The word "theek" (fine/okay) should mean neutral, and it does in the English translation ("fine" → Neutral, 78.43%), but when it is coupled with code, it somehow makes a significant positive prediction. This is a sign of harmful overconfidence—the model is very sure that it has made a wrong classification, which could lead to problems for applications that use it.

5.5.6 Summary of Error Patterns

The error study shows four major problems with XLM-R's ability to analyse sentiment across languages:

1. **Negation blindness (Test Groups 8, 10):** The model consistently fails to identify Nepali negation indicators ("bhayena," "naramro," "na-" prefix), frequently predicting the contrary sentiment with high confidence due to its tendency to overvalue positive content terms while disregarding negation (Arora, 2025).
2. **Unstable neutral class (Test Groups 12, 13, 14, 15):** Romanised Nepali neutral terms ("thikai," "milcha," "theek") are sometimes incorrectly categorised as Negative or Positive, indicating a lack of training data for neutral Nepali emotion or contradicting contextual associations acquired during pre-training (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).
3. **Code-mixing fragility (Test Groups 7, 9):** Code-mixed sentences don't work as well as monolingual ones (53.33% vs 60% Romanised Nepali vs 86.67% English accuracy), which shows that switching languages makes it harder for the model to understand compositional sentiment, especially when it comes to negation and neutral expressions (Yandrapati Prakash Babu, 2021).
4. **Confidence miscalibration (Test Groups 10, 12, 15):** The model shows severe overconfidence on faulty predictions, especially for Romanised Nepali and code-mixed inputs, with confidence ratings over 85-97% on improper classifications. This makes it less reliable for use in the actual world (Gibson Nkhata, 2025).

These results show that XLM-R is the best model overall, but it still has big language-specific biases and structural problems when it comes to processing Romanised Nepali-English code-mixed sentiment, especially when it comes to handling negation, finding neutral sentiment boundaries, and calibrating confidence across language variants.

2. Conclusion and Recommendations for Future Work

6.1 Summary of Findings

This dissertation examined sentiment analysis of code-switched Romanised Nepali-English text employing three deep learning architectures: BiLSTM with Attention, multilingual BERT (mBERT), and XLM-RoBERTa (XLM-R). The study investigated two principal research issues via methodical testing on YouTube comment data.

About RQ1 (Model Performance and Hyperparameter Optimisation):

At Epoch 2, all three models had their best validation performance. This shows that early halting is quite important in this low-resource situation. XLM-RoBERTa was the best architecture, although the performance metrics for each test set depended on the evaluation context:

1. The best hyperparameters for XLM-R are: Learning rate 3e-5, batch size 32, dropout 0.3, and early halting at Epoch 2-3 to stop overfitting (Sunil Gundapu, 2020).
2. Comparing the dynamics of training: XLM-R had the smallest gap between training and validation (around 0.19 by Epoch 3), BiLSTM+Attention had moderate overfitting (gap about 0.38 by Epoch 5), and mBERT had significant late-stage overfitting (validation loss up to 2.5 by Epoch 10) (Nusrat Jahan Prottasha, 2022).
3. Key insight: Multilingual pre-trained transformers, especially XLM-R with its large 100-language CommonCrawl training (2.5TB), do much better than BiLSTM trained from scratch. This shows how useful transfer learning is for code-mixed low-resource language pairs (Niraj Pahari, 2023).

On RQ2 (Cross-Lingual Consistency and Language Bais):

The cross-lingual consistency assessment utilising 15 semantically comparable sentence triplets (totalling 45 sentences) demonstrated notable language-dependent performance variations:

1. Systematic English bias: XLM-R got 86.67% of English variants right, but only 60% of Romanised Nepali and 53.33% of code-mixed text. There was a 26.7 percentage point disparity between English and Romanised Nepali (Yandrapati Prakash Babu, 2021).
2. In terms of weighted F1 differences, English (0.8611) did much better than Romanised Nepali (0.5493) and code-mixed variants (0.4815). When broken down by class, English got perfect positive class F1 (1.0), while Romanised Nepali and code-mixed had a hard time with neutral (0.3333, 0.4444) and negative (0.5455, 0.2857) classes (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).
3. Only 46.67% of triplets obtained full consistency, meaning that all three language varieties had the same correct predictions. This shows that cross-lingual robustness is moderate but not ideal (Aman Priuanshu, 2021).
4. Confidence miscalibration: Romanised Nepali variants had the highest average confidence (0.86) but the lowest accuracy (60%). English had lower confidence (0.82) but higher accuracy (86.67%). This shows that Romanised Nepali errors are too confident (Gibson Nkhata, 2025).
5. Patterns that were distinct to each class: Positive sentiment generalised well across variants, but negative sentiment had a lot of trouble being consistent. Code-mixed only got a negative F1 of 0.2857, whereas Romanised Nepali got a negative F1 of 0.5455 and English got a negative F1 of 0.75 (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

6.2 Research Contribution

This work advances the NLP research community's understanding of code-mixed sentiment analysis in several dimension.

1. **Systematic Architectural Comparison:** Conducted a thorough empirical comparison of BiLSTM+Attention, mBERT, and XLM-R on the same Romanised Nepali-English data, ensuring extensive hyperparameter adjustment for each architecture. The thorough examination of training dynamics, which looks at loss curves, overfitting patterns, and convergence behaviour, sets baseline performance expectations and the best training methods for this language pair that were not published in the literature before.
2. **Cross-Lingual Consistency Methodology:** Created a triplet-based evaluation methodology that uses semantically similar sentences in Romanised Nepali, English, and code-mixed forms to measure model robustness and find language-specific biases. This methodology transcends basic accuracy measurements to evaluate the stability of sentiment comprehension across linguistic variations of identical meaning, offering a framework for the assessment of multilingual sentiment systems in diverse code-mixed scenarios (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).
3. **Comprehensive Error Analysis:** Recorded systematic failure patterns particular to Romanised Nepali-English code-mixed emotion, encompassing:
    a. Negation blindness: The model doesn't see Nepali negation markers like "bhayena," "naramro," and the "na-" prefix, which leads to predictions that are the opposite of what they should be.
    b. Neutral class instability: Nepali neutral terms like "thikai" and "milcha" are often wrongly labelled as negative or positive.
    c. Code-mixing fragility: Performance decline on code-mixed variations compared to monolingual inputs, especially for sentences with negation.
    d. Confidence miscalibration: Overconfident wrong forecasts (85–97% confidence) on Romanised Nepali errors, which makes deployment less reliable.

These recorded patterns give useful information for making specific improvements to future code-mixed sentiment systems.

6.3 Limitations:

Dataset Scale: The dataset size is adequate for preliminary exploration and model comparison but is somewhat small compared to data-intensive deep learning methodologies, especially for encompassing the complete range of Romanised Nepali spelling variations and code-mixing patterns. More annotated corpora would make model training stronger, help find rare language phenomena (such sarcasm and complicated negation), and make cross-lingual consistency analysis more statistically powerful (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

Quality and Scope of Annotations: There were just a few annotators participating in the annotation process, even though it was done according to strict rules. Adding more people to the annotation team, including professional linguists who know how to express feelings in both Nepali and English, and doing multiple-pass annotation with formal adjudication procedures could make label reliability better and catch more subtle sentiment boundary cases, especially for the hard neutral class (Subalalitha Chinnaudayar Navaneethakrishnan, 2023).

Domain and Platform Specificity: The data was obtained solely from YouTube comments, reflecting informal social media communication. The results may not apply to other types of texts (such official news stories, product evaluations, or literary texts) or other social media sites (like Facebook or Twitter) where the rules for combining languages, showing emotions, and the types of people that use them are different. To see how portable the model is, it would need to be tested across different domains (Sanjeepan Sivapiran, 2023).

Script and Dialect Coverage: This study concentrated exclusively on Romanised Nepali alongside English, neglecting other significant variants: (Niraj Pahari, 2023)

- Devanagari Nepali-English code-mixing: How do models do when Nepali is written in its own script?
- Regional dialects: Nepali has many dialects (Eastern, Western, and Central) that may change how people express their feelings and how they write in Roman letters.
- Generalisability to Hindi-English, Bengali-English, and Tamil-English code-mixing in other South Asian language combinations has not been investigated.

Model Structure Scope: Due to computational and access constraints, experimentation was confined to models trainable on single-GPU infrastructure (BiLSTM, BERT-base, XLM-R-base). We did not investigate: (Gibson Nkhata, 2025)

- XLM-R-large and mBERT-large are two larger transformer variations that have more parameters.
- Generative LLMs: GPT-3.5, GPT-4, or other instruction-tuned models that can mix code by using few-shot prompting
- Pre-trained models that focus on South Asia, like IndicBERT, MuRIL, or other multilingual models

Interpretability Depth: Even though we looked at predictions, confidence ratings, and mistake patterns, the study didn't use any advanced interpretability methods.

- Attention visualisation: Looking at which tokens get a lot of attention for correct and erroneous predictions
- Using approaches like Integrated Gradients or GradCAM to find important input features is called gradient-based attribution.
- Probing tasks: Checking what the models have learnt about language (syntax, morphology, and negation) (Arora, 2025).

These methods might show why models don't work well with certain types of errors (such negation or neutral categorisation) and help with specific architectural changes.

6.4 Recommendation and Future Work

Short-Term Improvements (Immediate Implementation)

1. Targeted Data Growth: (Subalalitha Chinnaudayar Navaneethakrishnan, 2023)

Get more labelled examples with a focus on the following:

- Examples with a lot of negation: Sentences with "na," "bhayena," "chaina," and other Nepali negation words and sentiment words together
- Neutral sentiment expansion: More examples of Nepali neutral terms, such "thikai," "milcha," "theek," and "sakincha," to cut down on misclassification
- Negative class balancing: More examples of negative sentiment to fix the low consistency (28.57% F1 for code-mixed negative)
- Coverage of spelling variations: Different Romanisation forms of the same Nepali word (like "ramro," "ramri," and "ramree") with the same labels

2. Pipeline for Normalising Romanisation: (Supriya Chanda, 2024)

Use pre-processing to make Romanised Nepali spelling the same:

- Rule-based mapping: Change common variations to their standard forms, like "ramri" to "ramro" and "chha" to "cha."
- Edit-distance clustering: Put spelling variations together that are within a certain Levenshtein distance
- Phonetic encoding: Use Soundex or Metaphone-style techniques that have been changed to work with Nepali phonology.
- Learnt normalisation: Use parallel variant-standard pairs to train a character-level seq2seq model.

3. Fine-Tuning with Negation Awareness: (Arora, 2025)

Fix systematic negation failures by:

- Negation-augmented training: Make fake instances by adding or taking away negation markers from phrases that already have flipped labels.
- Regularisation of attention: Add loss terms that make negation markers get a lot of attention.
- Compositional pre-training: Before fine-tuning sentiment, train on a fake task that requires comprehending negation.

6.4 Final Remark

This research showed that XLM-RoBERTa works well for analyzing sentiment in mixed Nepali-English text written in Roman script, especially when you stop training early at the second epoch and tune the settings carefully. But the results also exposed some real problems the model got things right 86.67% of the time for English sentences but only 60% for Romanized Nepali, and barely half the test cases (46.67%) came out consistent across all three language versions. This gap tells us that even advanced multilingual models still favor widely-used languages and have trouble with less common writing styles, even when the underlying language is technically the same.

Looking closely at where things went wrong revealed several stubborn issues: the model often missed negative words in Nepali, struggled to pin down neutral feelings, got confused when languages mixed mid-sentence, and sometimes felt way too confident about predictions that turned out wrong. These findings make it clear that we can't consider sentiment analysis for mixed Nepali-English text a finished job, despite decent overall numbers. These problems need more work, especially since mixing languages while typing has become so common across South Asia and other regions.

The dataset we built, our three-way testing method, and the detailed breakdown of mistakes give other researchers something concrete to build on in this overlooked area. Millions of Nepali speakers and other South Asian users naturally write in mixed languages using Roman letters on social media every day. Making NLP tools that work just as reliably for them as they do for English - only users isn't just a tech problem to solve it's about fairness and making sure AI development includes everyone.

References

Abhash Pradhananga, A. K. S., 2023. Transformer-Based Deep Learning Models for Sentiment Analysis in Romanized Nepali: A comparative Investigation of BERT and RoBERTa, s.l.: ISSN.
Aman Priuanshu, A. V. S. S. S. V. N. C., 2021. ExCode-Mixed: Explainable Approaches towards Sentiment Analysis on Code-Mixed Data using BERT models, s.l.: s.n.
Anita Saroj, S. P., 2020. Sentiment Analysis on Multilingual Code Mixing Text using BERT-BASE participation of IRLab in Dravidian Code-mix and HASOC tasks of FIRE2020, s.l.: s.n.
Anon., 2020. Kaggle. [Online]
Available at: https://www.kaggle.com/code/vbookshelf/basics-of-bert-and-xlm-roberta-pytorch
Anon., n.d. Hugging Face. [Online]
Available at: https://huggingface.co/learn/llm-course/chapter6/6
Anuj Kumar, A. P. S. A. Y. P., 2022. On Enhancing Code-mixed sentiment and emotion classification using FNet and FastFormer, s.l.: s.n.
Arora, A., 2025. Sentiment Classification using Bi-LSTM with Attention.
Chahil Choudhary, J. T. S., 2024. Sentiment Analysis in Code-Mixed Video Comments Using Machine Learning on a Hinglish Dataset, s.l.: s.n.
Das, S., 2021. Sentiment Recognition of Hinglish Code Mixed Data using Deep learning Model based approach, s.l.: s.n.
Dou Hu, L. W. Y. Z. S. H., 2023. UCAS-IIE-NLP at SemEval-2023 Task 12: Enhancing Generalization of Multilingual BERT for Low-Resource Sentiment Analysis.
Gibson Nkhata, S. G., 2025. Fine-tuning BERT with Bidirectional LSTM for Fine-grained Movie Reviews Sentiment Analysis, s.l.: s.n.
Hu, D. H. L. W. Y. L. W. Z. S., 2023. UCAS-IIE-NLP at SemEval-2023 Task 12: Enhancing Generalization of, s.l.: s.n.
Manish Pandey, N. P. y. M. A., 2025. Creating and Evaluating Code-Mixed Nepali-English and Telugu-English Datasets for Abusive Language Detection Using Traditional and Deep Learning Models, s.l.: s.n.
Naeem, A., n.d. Sentiment Analysis using LSTM, TensorFlow, and Keras. [Online]
Available at: https://www.embedded-robotics.com/sentiment-analysis-using-lstm/
Nikita Singhal, a. Y. A. G. S. R. K., 2025. Leveraging XLM-RoBERTa with CNN and BiLSTM for Hinglish Toxicity Detection, s.l.: s.n.
Niraj Pahari, K. S., 2023. Character-level Data Augmentation on Code Mixed Sentences for Low-Resource Settings, s.l.: s.n.

Niraj Pahari, K. S., 2023. Language Perference for Expression of Sentiment for NepaliEnglish Bilingual Speaker on Social Media, s.l.: s.n.
Nusrat Jahan Prottasha, A. A. S. M. K. S. A. M. A. K. B. M. M. a. M. B., 2022. Transfer Learning for Sentiment Analysis Using BERT Based Supervised Fine-Tuning. MDPI.
Pahari, N. S. K., 2023. Language Preference for Expression of Sentiment for {N}epali-{E}nglish Bilingual Speakers on Social Media",, s.l.: s.n.
Pranaydeep Singh, E. L., 2020. Sentiment Analysis for Hinglish Code-mixed Tweets by means of cross-lingual word embeddings, s.l.: s.n.
Rajesh Piryani, B. P. V. K. S. D. P., 2020. Sentiment analysis in Nepali: Exploring machine learning and lexicon-based approaches, s.l.: s.n.
Rauniyar, R., 2025. Rebulica Nagarik. [Online]
Available at:
https://myrepublica.nagariknetwork.com/index.php/news/use-of-nepali-language-rises-sharply-over-two-decades-69-76.html#:~:text=The%202001%20national%20census%20showed%20that%2025.2%20percent%20of%20the%20population%20spoke%20Nepali%20as%20a%20second%20lan
Sadia Alam, M. F. I. N. H. A., n.d. BNSENTMIX: ADiverse Bengali-English Code-Mixed Dataset for sentiment analysis, s.l.: s.n.
Sameer Sitoula, T. B. S. W. A. N., 2025. Sentiment analysis of Nepali social media text with a hybrid deep learning Model, Kathmandu: Springer.
Sanjeepan Sivapiran, C. V. a. U. T., 2023. Sentiment Analysis in Dravidian Code-Mixed YouTube Comments and Post, s.l.: s.n.
Shuyue Stella Li, K. M., 2022. Language Agnostic Code-Mixing Data Augmentation by Predicting Linguistic Patterns, s.l.: arXiv.
Subalalitha Chinnaudayar Navaneethakrishnan, B. R. C. K. S. M. S. P. K. K. B. L. S. K. R. P., 2023. Findings of shared task on Sentiment Analysis and Homophobia Detection of YouTube Comments in Code-Mixed Dravidian Languages, s.l.: s.n.
Sunil Gundapu, R. M., 2020. SemEval 2020 Task 9: Syntactic Semantic LSTM Architecture for Sentiment Analysis of Code-mixed data, s.l.: s.n.
Supriya Chanda, A. M. ,. S. P., 2024. Sentiment analysis of code-mixed Dravidian languages leveraging pretrained model and word-level language tag, s.l.: s.n.
Yandrapati Prakash Babu, E. R., 2021. Sentiment Analysis on Dravidian Code-Mixed YouTube Comments using Paraphrase XLM-RoBERTa Model, s.l.: s.n.

Yihan Deng, J. v. d. m. A. T. M. S. C. B. K. D., 2025. Analyzing Sleep Behavior Using BERT-BiLSTM and Fine-Tuned GPT-2 Sentiment Classification: Comparison Study, s.l.: s.n.

Appendices

Appendix A: Dataset Annotation Guidelines

Definitions of sentiment classes:

- Positive: Shows enjoyment, satisfaction, agreement, praise, or a good opinion.
  - "This is amazing!" "I love this!" "Timi kati ramri!"
- Negative: Shows anger, disappointment, criticism, sadness, or a bad opinion.
  - For example, "Terrible experience," "I hate this," and "Yo naramro chha."
- Neutral: Statements of fact or a lack of definite feelings.
  - "The meeting is at 3 PM," "The weather is cloudy," and "Meeting aaile 3 baje cha" are all examples.

Ambiguous cases: - Sarcasm: Treat based on literal interpretation, or if clearly sarcastic, reverse the polarity. - Questions: Classify based on implied sentiment, or as neutral if no sentiment is clear. - Emojis: Consider emoji sentiment alongside text.

Appendix B: Hyperparameter Tuning Full Results

B.1 BiLSTM with Attention – Complete Grid Search Results

The following table present 4 hyperparameter configuration tested during the BiLSTM + Attention grid search, sorted by validation weighted F1-Score.

S.N.	Batch_Size	Dropout	Hidden Dim	Learning rate	Epoch	Weight Decay	Precision	Recall	F1-Score
1	32	0.3	256	1e-3	20	0	0.68	0.67	0.67
2	32	0.5	128	1e-3	6	1e-4	0.70	0.68	0.68
3	32	0.6	128	1e-3	3	1e-4	0.71	0.67	0.67
4	32	0.7	128	1e-3	3	1e-4	0.69	0.68	0.68

Key Observations:

- Best Configuration (4)
- Dropout sweet spot: 0.7 dropout provide best regularization balance
- Balanced Model, with no overfitting like others.

B.2 Mulitlingual BERT (mBERT) – Completed Tuning Results

S.N.	Max_length	Batch_Size	Dropout	Warm up ratio	Lr (Encoder)	Lr (Classifier)	Epoch	Weight Decay	Precision	Recall	F1-Score
1.	192	16	0.1	0.05	1e-5	5e-4	10	0.01	0.73	0.72	0.72
2.	192	32	0.3	0.1	5e-6	2e-4	10	0.02	0.69	0.70	0.69
3.	256	16	0.15	0.06	1.5e-5	4e-4	10	0.01	0.72	0.71	0.71

Key Observations:

- Best Configuration (3)
- Optimal sequence is 256 tokens.

B.3 XLM-RoBERTa (XLM-R) – Complete Tuning Results

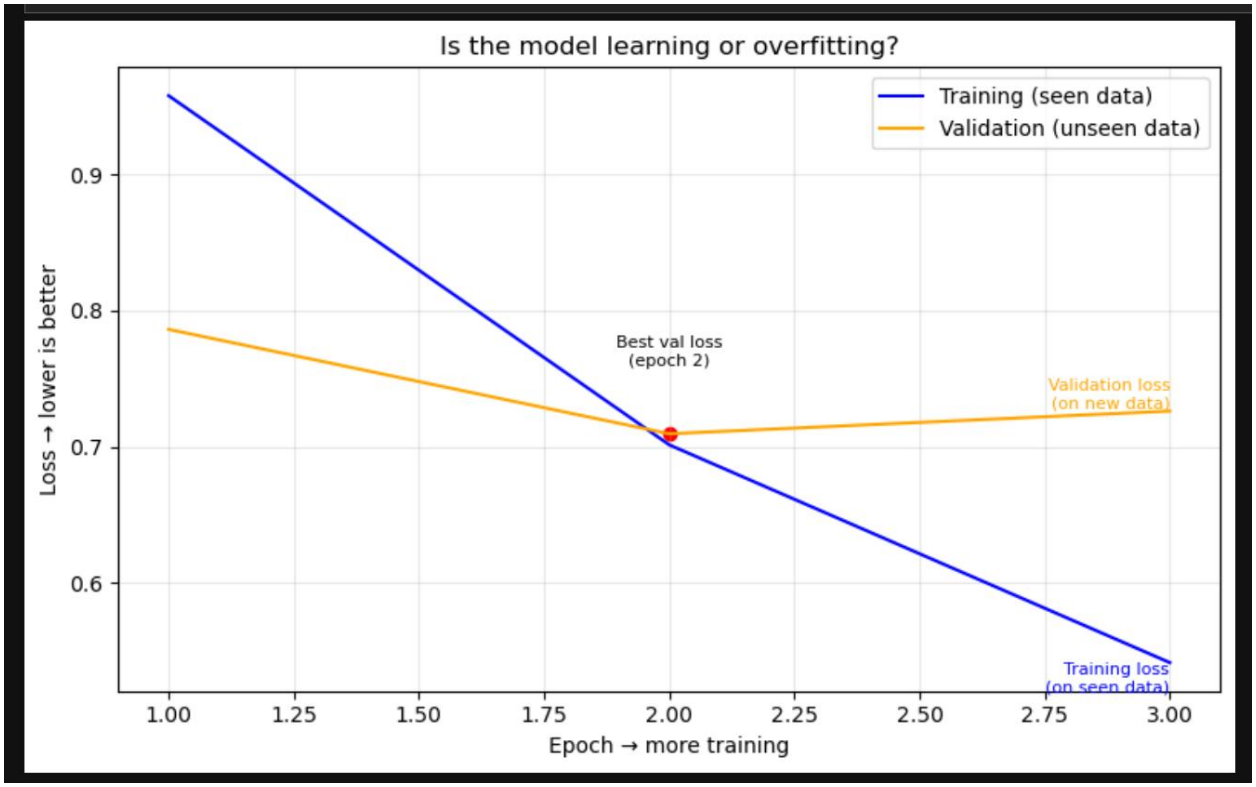
S.N.	Max_length	Batch_Size	Dropout	Warm up ratio	Lr (Encoder)	Lr (Classifier)	Epoch	Weight Decay	Precision	Recall	F1-Score
1.	128	16	0.3	0.1	2e-5	1e-4	5	0.01	0.75	0.75	0.75
2.	256	32	0.4	0.15	3e-5	2e-4	7	0.15	0.74	0.74	0.74
3	128	32	0.4	0.04	2e-5	1e-4	3	0.04	0.72	0.72	0.72

Key Observation:

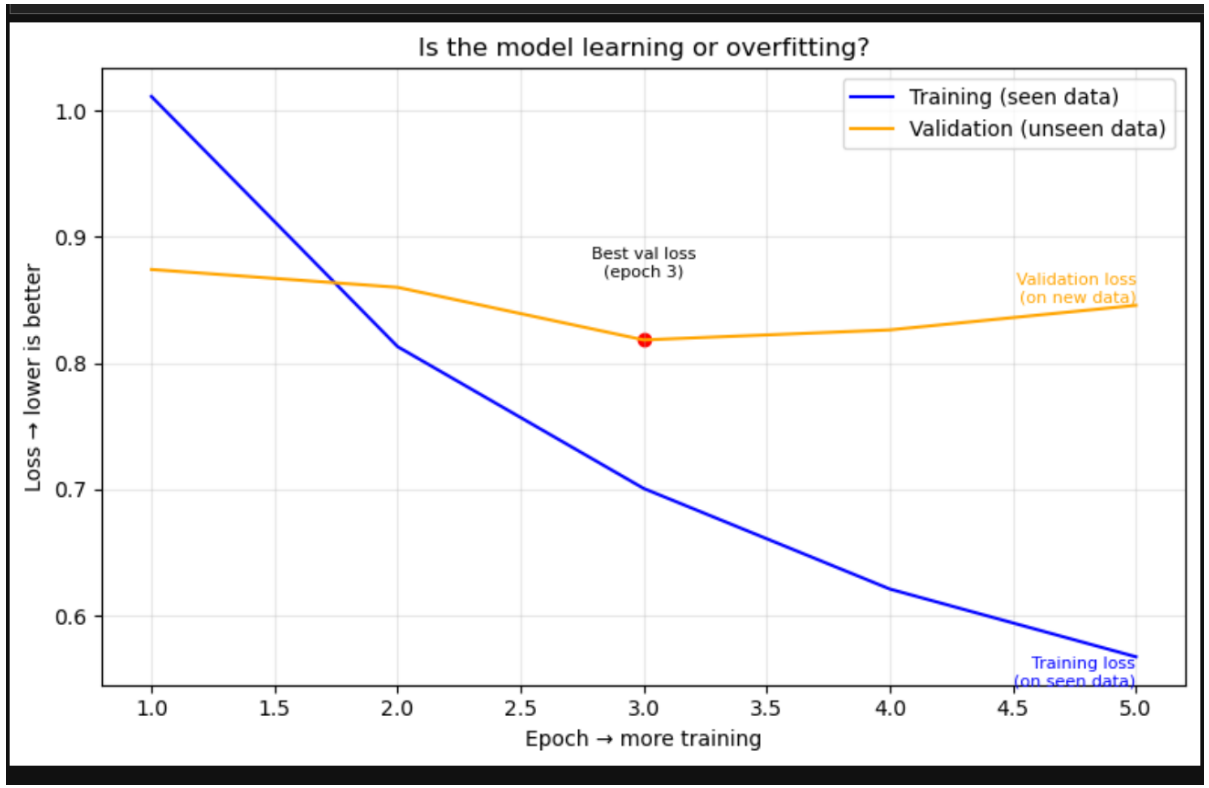
- Best Configuration (3)
- Combination of Dropout (0.4) and 2e-5 learning rate is optimal for preventing overfitting.

Appendix C: Loss Curves of All Models

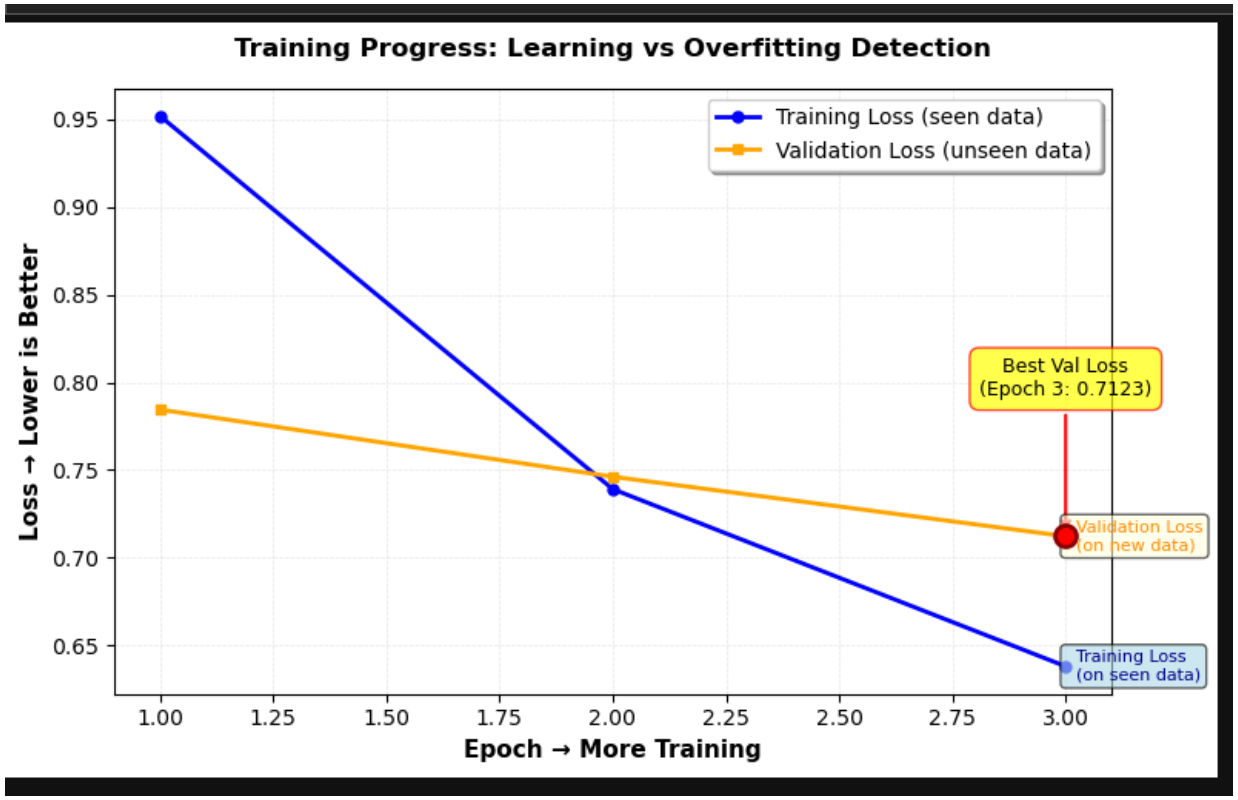
C.1 BiLSTM with Attention – Training Dynamics



C.2 mBERT Training and Validation Curves

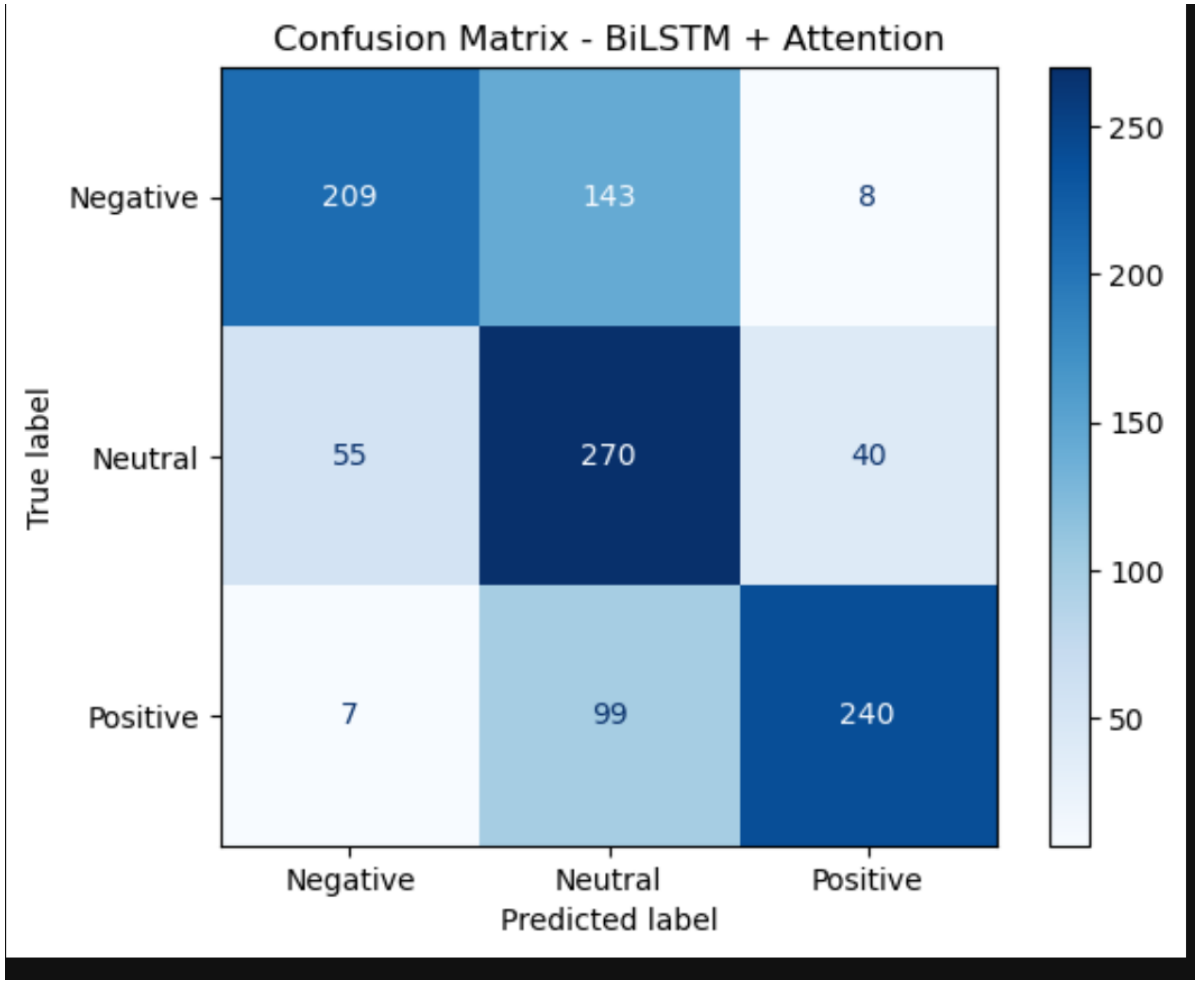


C.3 XLM-R Training and Validation Curves

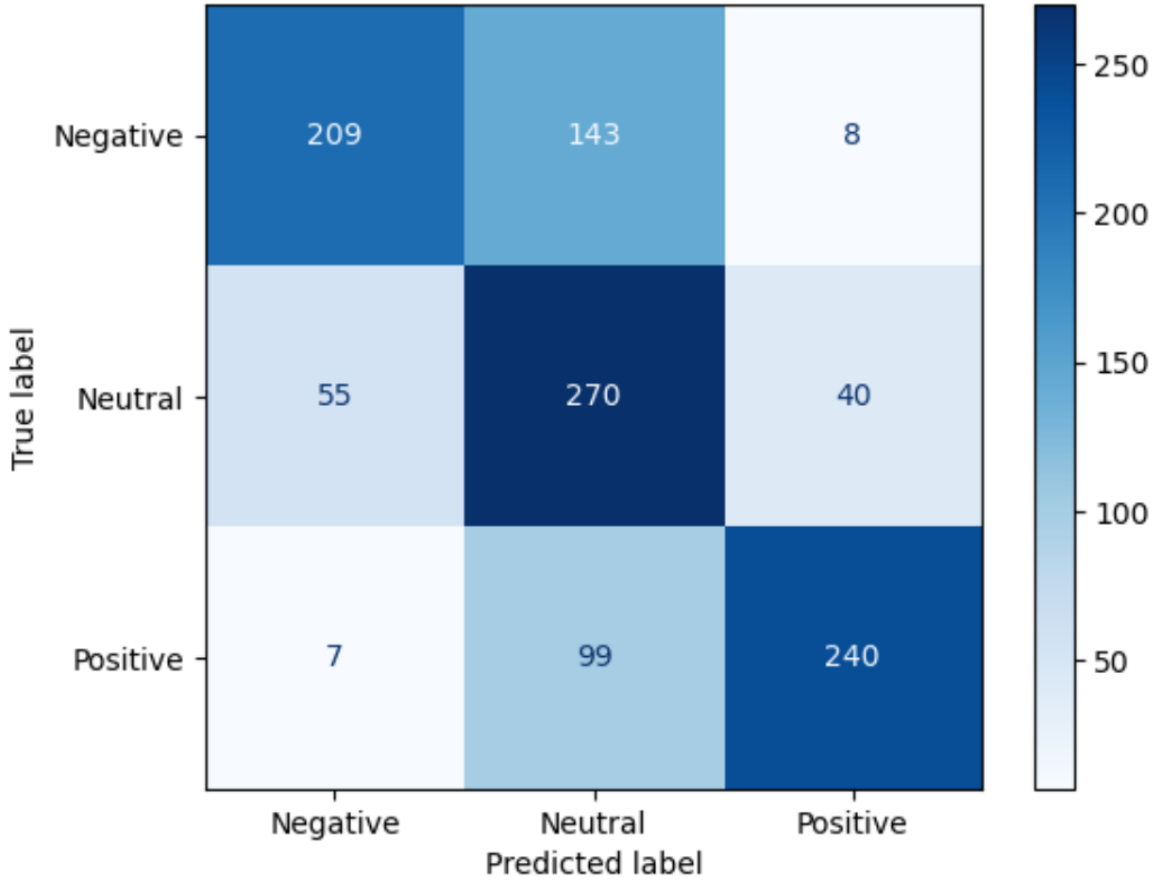


Appendix D: Confusion Matrices

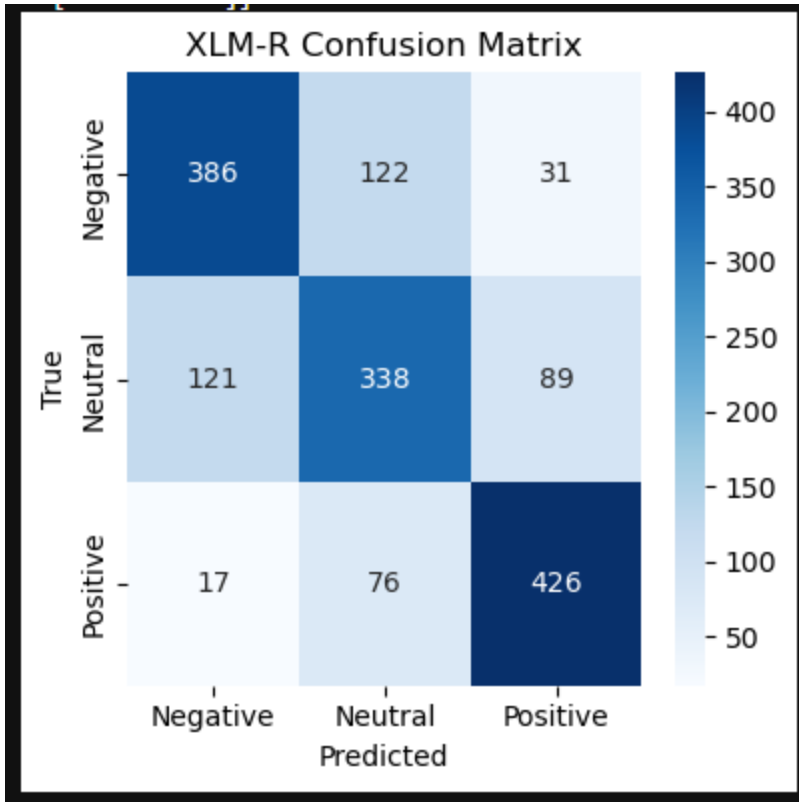
D.1 BiLSTM with Attention Confusion Matrix



D.2 mBERT Confusion Matrix

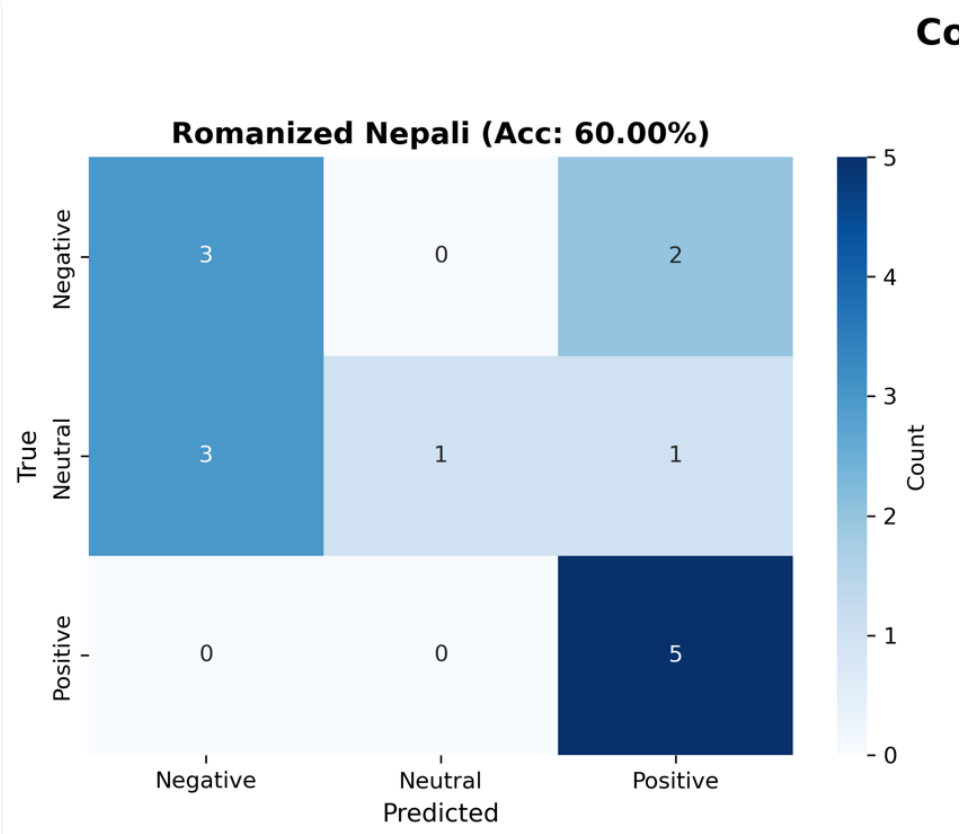


D.3 XLM-R Confusion Matrix

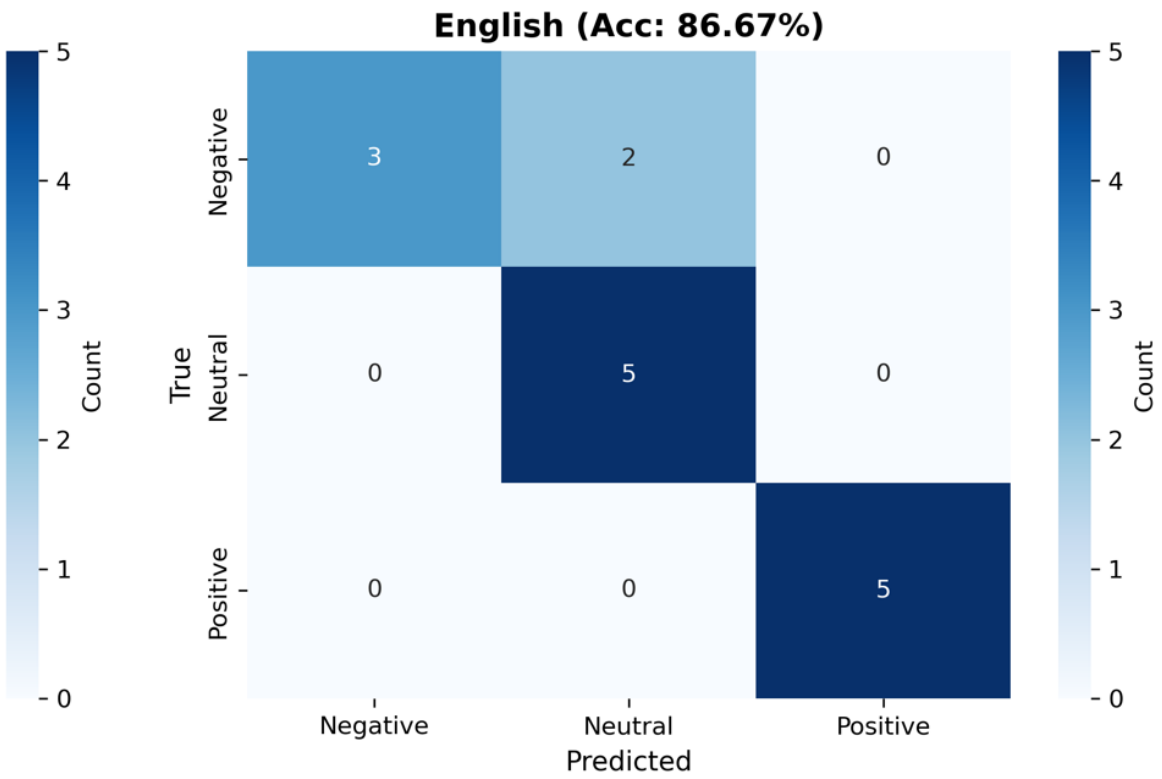


D.4 Cross-Lingual Consistency – XLM-R Per-Variant Confusion Matrices

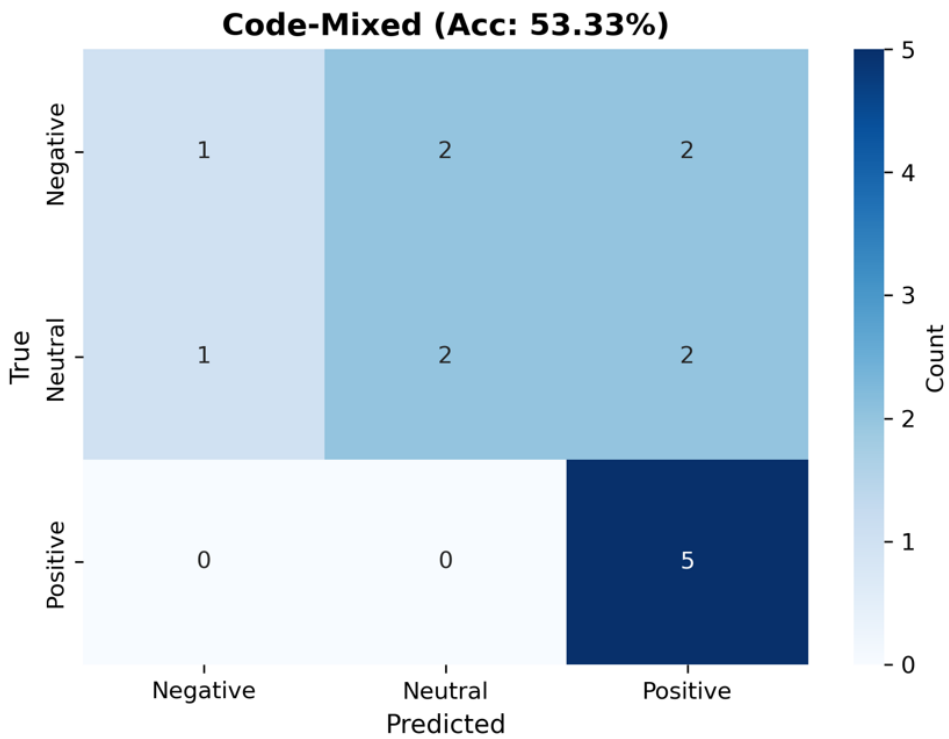
Romanized Nepali Variant



English Variant:



Code-Mixed Variant



Appendix E: Logbook

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 01 **Date: 2025/11/17**

Start Time: 8:00 PM **End Time: 8:50 PM**

Items Discussed:
We discussed the overall scope of the dissertation project. My supervisor advised removing the traditional NLP and basic machine learning models from the comparison list and focusing solely on deep learning-based models such as Seq2Seq, RNN, CNN, and Transformer and Hybrids models. We also reviewed the dataset selection. The supervisor recommends using a more complex code-mixed dataset to better test the performance and robustness of these models.

Achievements:
I gained valuable insights and a clear direction for the dissertation based on the supervisor's feedback.

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)
1. Research Deep-Learning based NLP models in depth.
2. Explore and analyze complex, code-mixed datasets suitable for the study.
3. Review research articles on code-mixed sentiment analysis.

Student Sign **External Supervisor Sign**

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 02 **Date: 2025/11/24**

Start Time: 8:00 PM **End Time: 8:40 PM**

Items Discussed:
We reviewed the model selection approach and discussed the next steps, including finalizing the selected models, establishing baseline performance metrics for each, model conducting further literature review and creating a matrix map for proper documentation.

Achievements:
I gained valuable insights and a more structured direction for the model selection process.

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)
Review all research conducted on selected models and related research papers.

Student Sign **External Supervisor Sign**

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 03 **Date: 2025/12/02**

Start Time: 8:00 PM **End Time: 8:40 PM**

Items Discussed:
In our third meeting, we reviewed the finalized model selection. Based on the supervisor's guidance, we confirmed the models to be used for the research. We Also defined the research scope and problem statement. Additionally, we examined the dataset and planned the work to be carried out during the upcoming week.

Achievements:
I gained a new perspective on how to approach the research process and structure my work effectively.

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)
1. Work on the dataset preparation.
2. Begin benchmarking the selected models simultaneously.

Student Sign **External Supervisor Sign**

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 04 **Date: 2025/12/08**

Start Time: 8:20 PM **End Time: 9:00 PM**

Items Discussed:
In our fourth meeting, we discussed the extended scope of the dataset that will be used to test our models. We agreed to focus on the dataset for the next few weeks to ensure a thorough and accurate analysis.

Achievements:
We outlined and discussed the Exploratory Data Analysis (EDA) process.

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)
1. Continue working on the dataset and carry out the Exploratory Data Analysis (EDA)

Student Sign **External Supervisor Sign**

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 05 **Date: 2025/12/02**

Start Time: 8:00 PM **End Time: 8:35 PM**

Items Discussed:
In our fifth meeting, we reviewed the model performance, including accuracy metrics and the number of parameters used during training. We also assessed the progress of the data annotation work. Additionally, we discussed possible ways to improve model performance by tuning hyperparameters and adjusting model configurations.

Achievements:
We identified and discussed strategies for tuning hyperparameters to improve model accuracy

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)
1. Continue researching hyperparameter tuning, overfitting, and underfitting.
2. Continue data annotation work.
3. Test the model under different scenarios, including sentence variations, spelling variations, and other input inconsistencies.

Student Sign **External Supervisor Sign**

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 06 **Date: 2026/01/05**

Start Time: 8:15 PM **End Time: 8:35 PM**

Items Discussed:
In our sixth meeting, we reviewed the progress of my assigned tasks. We discussed in detail the strategies used to tweak and optimize hyperparameter in order to improve model performance. This included adjusting learning rates, batch sizes, number of epochs, hidden layer sizes, and regularization techniques to achieve better generalization and reduce overfitting.
Additionally, we discussed the second research question and its alignment with the ongoing experiments and analysis.

Achievements:
We outlined the approach for addressing and answering the research questions and also discussed the structure and expectations for report writing.

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)
1. Continue researching hyperparameter tuning, overfitting, and underfitting.
2. Continue data annotation work.
3. Test the model under linguistic, contextual, and methodological challenges..

Student Sign **External Supervisor Sign**

MSc Project Meeting Logbook Entry Sheet

Meeting Logbook Entry Sheet

Meeting No: 07 **Date: 2026/01/12**

Start Time: 8:05 PM **End Time: 8:45 PM**

Items Discussed:
In our seventh meeting, we reviewed the progress of my assigned tasks. We discussed the completion of the previous week's tasks, including improvements made to the trained models through hyperparameter tuning. I also shared my progress on the dissertation report, which was reviewed and discussed during the meeting.

Achievements:
We outlined the remaining steps for completing the dissertation report and further optimizing the trained models.

Problems:
No problem was encountered during the meeting.

Tasks for Next Meeting: (Write down the task assigned to you by your supervisor.)
1. Continue working on model tuning and optimization.
2. Continue dissertation report writing.

Student Sign **External Supervisor Sign**